



University of
Salford
MANCHESTER

ADVANCED DATABASE

SG-G430-M0052-33386-26

BRIGHT OZUZU
UZOSIKE

STUDENT ID
@00808384

Table of Contents

Table of Contents.....	1
Online Banking Database: Design, Implementation & Management.....	3
Task 1 Report: Online Banking Database: Design, Implementation & Management. .	4
Introduction.....	4
Part 1: Database Design and Normalisation.....	5
1.1 Identifying Entities and Relationships.....	5
1.2 Normalisation to Third Normal Form.....	6
1.3 Entity-Relationship Diagram.....	6
1.4 Table Creation and Data Types.....	7
Constraints Applied	8
1.5 Populating the Database.....	14
Part 2: Database Objects and Queries.....	16
2.1 Searching any Accounts by Name.....	16
2.2 Payments Which Are Due in Less Than 5 Days.....	18
2.3 Inserting New Customers.....	20
2.4 Updating an Existing Customer.....	24
2.5 Snapshot View of Transactions Having Overdue Fees.....	27
2.6 Trigger for Closing an Account Automatically.....	29
2.7 Customers Having Less Than 50% Overdue Fees Repayment.....	32
2.8 Test Cases of Database Objects.....	33
2.9 Some Additional Database Objects.....	33
7a: Customer Account Summary Procedure	33
7b: Customer Archival Trigger	35
Additional Recommendations.....	38
Data Integrity and Concurrency.....	38
Database Security.....	38
Database Backup and Recovery.....	39
Conclusions.....	40
NHS Prescribing Data: SQL Querying & Analysis.....	41
Task 2 Report: NHS Prescribing Data: SQL Querying & Analysis.....	42
Introduction.....	42
Part 1: Database Creation and Data Import.....	43
1.1 Creating the Database.....	43
1.2 Importing the CSV Files.....	43
1.3 Adding Primary and Foreign Key Constraints.....	52

1.4 Database Diagram.....	52
1.5 Verifying the Import.....	53
Part 2: Queries and Analysis.....	54
2.1 Question 2: Drugs in Tablet or Capsule Form.....	54
2.2 Question 3: Total Quantity per Prescription.....	54
2.3 Question 4: Most Prescribed Chemical Substance per Month.....	55
2.4 Question 5: Prescriptions per BNF Chapter with Cost Statistics.....	56
2.5 Question 6: Most Expensive Prescription per Practice (Over £4,000).....	57
2.6 Question 7: Additional Queries.....	58
7a: Practice Specialisation Analysis.....	58
7b: Bulk Purchasing Opportunities.....	59
7c: Unusual or Potentially Erroneous Prescriptions.....	60
7d: Month-on-Month Comparison of Prescribing Volumes.....	61
Conclusions.....	64
References.....	65

Online Banking Database: Design, Implementation & Management

Task 1 Report: Online Banking Database: Design, Implementation & Management

Introduction

The design and implementation of a relational database system for an online banking platform is documented in this report. The bank requires a system capable of managing customer information, account details, transaction records, overdue fee tracking, and repayment processing. The platform handles hundreds of transactions daily, making data integrity, consistency, and performance critical considerations throughout the design process.

The approach taken in this report follows a top-down database design methodology. Starting from the client requirements specification, the key entities were identified and their relationships mapped. The schema was then normalised to Third Normal Form (3NF) to eliminate redundancy and prevent insertion, update, and deletion anomalies. All tables, constraints, and database objects were implemented using T-SQL in Microsoft SQL Server Management Studio (SSMS) 2022.

Beyond the core requirements, the design incorporates forward-looking features such as support for joint accounts through a many-to-many relationship model, password hashing using SHA2_512 with unique salts for each user, and a customer archival mechanism for marketing data retention. These additions reflect the kind of flexibility that a production banking system would demand as requirements evolve over time. The report is structured as follows: Part 1 covers the database design, normalisation, and table creation. Part 2 addresses the stored procedures, functions, views, triggers, and queries required by the brief. Finally, recommendations are provided on data integrity, concurrency, security, and backup and recovery, all applied specifically to this online banking scenario.

Part 1: Database Design and Normalisation

1.1 Identifying Entities and Relationships

The first step in the design process was to carefully read the client requirements and extract the entities and their attributes. Following the principles covered in the database design lecture, nouns in the specification were treated as candidate entities or attributes, while verbs were treated as candidate relationships.

The following core entities were identified from the requirements specification:

Customer - Stores personal details (name, date of birth), authentication credentials (username, password), and optional contact information (email, telephone). The brief states that customer information is retained even after account closure for marketing purposes, so an `IsActive` flag is used rather than deleting records.

Address - Contains the physical address fields (`AddressLine1`, `AddressLine2`, `City`, `Postcode`). This was separated from `Customer` to achieve Third Normal Form, since address fields are not functionally dependent on `CustomerID`. Two customers living at the same address (for example, family members) would otherwise cause data redundancy.

AccountType – This table had the five account types specified in the brief: Savings, Checking, Loan, Credit Card, and Investment. The type name is functionally dependent on the type itself, not on any individual account, so extracting it into a separate table follows the same normalisation principle used in the `SalfordBankLtd` workshop.

Account - Represents each bank account or product. Holds the account name, status (`Active`, `Dormant`, `Closed`, `Frozen`), opening date, closure date, balance, and an optional reference number for loan-type products.

CustomerAccount – This table was used to resolve the many-to-many relationship between the `Customer` table and the `Account` table. While the brief does not explicitly mention joint accounts, this design provides the flexibility to support them in future without structural changes to the schema. This follows the same pattern as the `CustomerAccounts` table in the Week 7 workshop.

Transaction - Records each banking transaction, linked to both an account and the customer who initiated it. Includes transaction date, due date (for loan or credit payments), and completion date (`NULL` if the transaction is still pending).

OverdueFee - Tracks fees charged for late payments. Each overdue fee is linked to a specific transaction and records the fee amount, the number of days overdue, the total owed, and the total repaid. The outstanding balance is implemented as a computed column (`TotalOwed` minus `TotalRepaid`), ensuring it is always mathematically consistent.

Repayment - Records individual repayments made by customers toward their overdue fees. Each repayment is linked to a specific `OverdueFee` record and captures the date, amount, and payment method (`Bank Transfer`, `Card`, or `Cash` as specified in the brief).

1.2 Normalisation to Third Normal Form

The database was designed to satisfy Third Normal Form (3NF). According to Elmasri and Navathe (2016), a relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key. The normalisation decisions are documented below.

First Normal Form (1NF): All tables contain only atomic values with no repeating groups. Each column holds a single value, and each row is uniquely identifiable by its primary key. For example, the Customer table stores first name and last name in separate columns rather than a single combined name field.

Second Normal Form (2NF): All non-key attributes are fully functionally dependent on the entire primary key. A case for this is seen in the CustomerAccount table. The table serves as a bridge between the Customer and Account tables, having a combination of CustomerID and AccountID columns as its primary key. The DateLinked attribute depends on the combination of both keys (composite key), not on either individually.

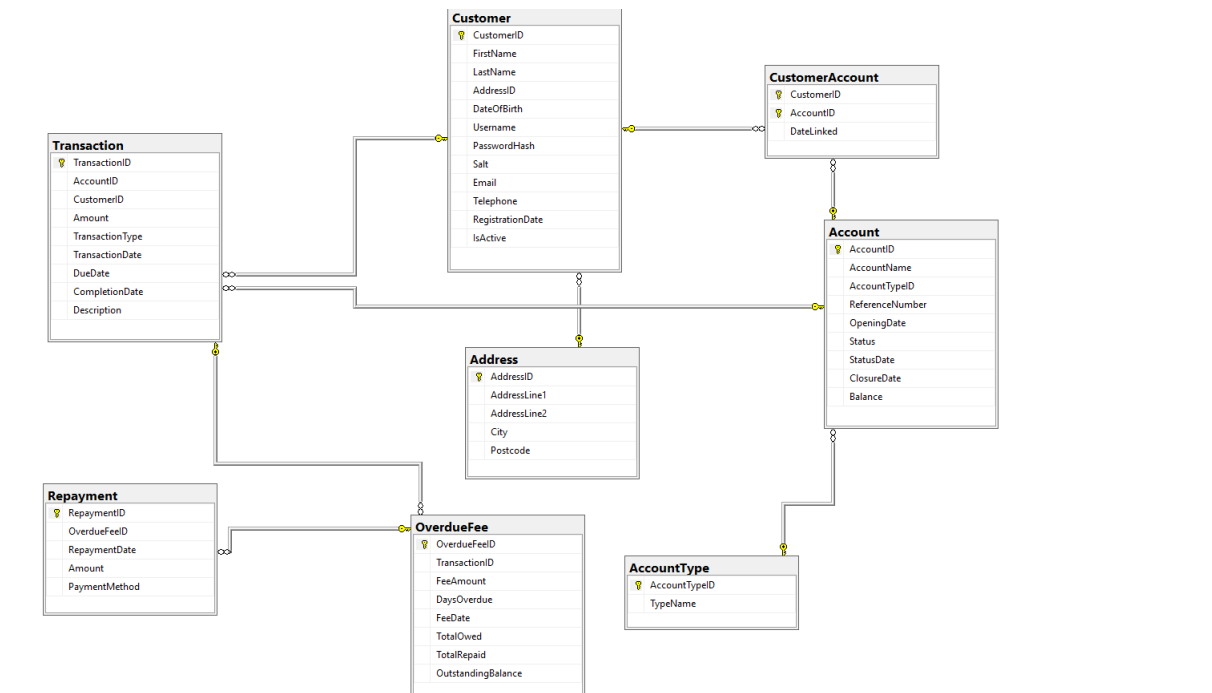
Third Normal Form (3NF): No transitive dependencies exist. Two key normalisation decisions were made to achieve this:

First, Address was separated from Customer. In the original flat structure, address fields such as City and Postcode are functionally dependent on AddressLine1 and Postcode (not on CustomerID). Keeping them in the Customer table would create a transitive dependency: CustomerID determines AddressID, and AddressID determines City and Postcode. By extracting addresses into their own table with a UNIQUE constraint on (AddressLine1, Postcode), this transitive dependency is removed.

Second, AccountType was extracted as a lookup table. If the type name were stored directly in the Account table, changing the name of an account type (for instance, renaming 'Checking' to 'Current Account') would require updating every row. The lookup table means the name is stored once and referenced via a foreign key, eliminating this update anomaly.

1.3 Entity-Relationship Diagram

The diagram below shows all eight tables and their relationships, including the primary keys, foreign keys, and the cardinality of each relationship.



The key relationships are: Address to Customer (one-to-many), Customer to CustomerAccount (one-to-many), Account to CustomerAccount (one-to-many, forming the many-to-many link), AccountType to Account (one-to-many), Account to Transaction (one-to-many), Customer to Transaction (one-to-many), Transaction to OverdueFee (one-to-zero-or-one), and OverdueFee to Repayment (one-to-many).

1.4 Table Creation and Data Types

T-SQL statements were employed in creating all tables in the database. The tables are created in dependency order so that foreign key references always point to tables that already exist. For cases where the table names to be given were considered reserved by SMSS, '[' was used to escape it (the only case was with Transaction table). The full script is included in the accompanying SQL file. Below is a summary of each table with the reasoning behind the key design choices.

Column	Data Type	Justification
CustomerID, AccountID, etc.	INT IDENTITY(1,1)	Auto-incrementing surrogate keys provide unique identification without relying on business data that could change.
FirstName, LastName	NVARCHAR(50)	NVARCHAR supports Unicode characters for international names. 50 characters provides reasonable capacity without wasting storage.
PasswordHash	BINARY(64)	Fixed-length binary output from SHA2_512 hashing. Plain-text passwords are never stored, following security best practice.
Salt	UNIQUEIDENTIFIER	Generates a globally unique salt for each customer, preventing rainbow table attacks.

Column	Data Type	Justification
Balance, Amount	DECIMAL(18,2)	Exact numeric type for monetary values. Preferred over MONEY because DECIMAL provides explicit control over precision and scale and avoids implicit rounding during division operations.
DateOfBirth, OpeningDate	DATE	Stores date only without time component, which is appropriate for dates where time is irrelevant.
TransactionDate	DATETIME2	Captures both date and time with higher precision than the legacy DATETIME type. Important for recording the exact moment of financial transactions.
Telephone	NVARCHAR(20)	Phone numbers can contain spaces, plus signs, and leading zeros, so they must be stored as character strings, not numeric types.
IsActive	BIT	Boolean flag (0 or 1) to indicate whether a customer's relationship with the bank is active. Supports the requirement to retain closed customer records for marketing.
OutstandingBalance	Computed Column	Defined as (TotalOwed - TotalRepaid). Automatically calculated, ensuring mathematical consistency without manual updates.

Constraints Applied

Several constraints were applied to enforce data integrity at the database level, reducing the reliance on application-level validation. These include:

PRIMARY KEY constraints on all tables to uniquely identify each row. FOREIGN KEY constraints to enforce referential integrity between related tables, ensuring that, for example, a transaction cannot reference a non-existent account. CHECK constraints to restrict column values to valid business options, such as limiting account status to 'Active', 'Dormant', 'Closed', or 'Frozen', and payment methods to 'Bank Transfer', 'Card', or 'Cash'. A CHECK constraint on the Email column validates the basic format of email addresses using a LIKE pattern. A UNIQUE filtered index on ReferenceNumber ensures that loan and credit card reference numbers are unique when provided, while allowing multiple NULL values for accounts that do not require a reference number. The DEFAULT constraint on several columns (such as Status defaulting to 'Active' and Balance defaulting to 0.00) reduces the risk of incomplete data entry.

```

CREATE TABLE dbo.Address (
    AddressID    INT IDENTITY(1,1) NOT NULL,
    AddressLine1 NVARCHAR(100)     NOT NULL,
    AddressLine2 NVARCHAR(100)     NULL,
    City         NVARCHAR(50)      NOT NULL,
    Postcode     NVARCHAR(10)      NOT NULL,

    CONSTRAINT PK_Address PRIMARY KEY (AddressID),

    -- In places like the UK, AddressLine1 + Postcode uniquely identifies
    -- a property, preventing duplicate address records.
    CONSTRAINT UC_Address UNIQUE (AddressLine1, Postcode)
);

CREATE TABLE dbo.Customer (
    CustomerID    INT IDENTITY(1,1) NOT NULL,
    FirstName     NVARCHAR(50)      NOT NULL,
    LastName      NVARCHAR(50)      NOT NULL,
    AddressID     INT                NOT NULL,
    DateOfBirth   DATE              NOT NULL,
    Username      NVARCHAR(50)      NOT NULL,
    PasswordHash  BINARY(64)        NOT NULL,
    Salt          UNIQUEIDENTIFIER  NOT NULL,
    Email         NVARCHAR(100)      NULL,    -- Optional
    Telephone     NVARCHAR(20)       NULL,    -- Optional
    RegistrationDate DATETIME2      NOT NULL DEFAULT GETDATE(),
    IsActive      BIT               NOT NULL DEFAULT 1,

    CONSTRAINT PK_Customer PRIMARY KEY (CustomerID),

    -- Username must be unique across all customers
    CONSTRAINT UQ_Customer_Username UNIQUE (Username),

    -- CHECK constraint to validate email format when provided
    -- This same pattern was used in Week 7 workshop (SalfordBankLtd). Applying it here
    CONSTRAINT CHK_Customer_Email CHECK (
        Email IS NULL OR Email LIKE '%_@_%._%'
    ),

    -- DOB sanity check: customer must be at least 16 years old to comply with
    -- account opening legal regulations.
    CONSTRAINT CHK_Customer_DOB CHECK (
        DateOfBirth <= DATEADD(YEAR, -16, GETDATE())
    ),

    -- Foreign key to Address table
    CONSTRAINT FK_Customer_Address FOREIGN KEY (AddressID)
        REFERENCES dbo.Address(AddressID)
);

```

```

CREATE TABLE dbo.AccountType (
    AccountTypeID INT IDENTITY(1,1) NOT NULL,
    TypeName NVARCHAR(50) NOT NULL,

    CONSTRAINT PK_AccountType PRIMARY KEY (AccountTypeID),
    CONSTRAINT UQ_AccountType_Name UNIQUE (TypeName),

    -- Data validation check for the five types of account type specified
    CONSTRAINT CHK_AccountType_Name CHECK (
        TypeName IN ('Savings', 'Checking', 'Loan', 'Credit Card', 'Investment')
    )
);

CREATE TABLE dbo.Account (
    AccountID INT IDENTITY(1,1) NOT NULL,
    AccountName NVARCHAR(100) NOT NULL,
    AccountTypeID INT NOT NULL,
    ReferenceNumber NVARCHAR(50) NULL, -- For loans; NULL for others
    OpeningDate DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
    Status NVARCHAR(20) NOT NULL DEFAULT 'Active',
    StatusDate DATETIME2 NULL, -- When status last changed
    ClosureDate DATE NULL, -- NULL if not closed
    Balance DECIMAL(18,2) NOT NULL DEFAULT 0.00,

    CONSTRAINT PK_Account PRIMARY KEY (AccountID),

    -- Another data validation check to restrict status to the four values specified in the brief
    CONSTRAINT CHK_Account_Status CHECK (
        Status IN ('Active', 'Dormant', 'Closed', 'Frozen')
    ),

    -- Foreign key to AccountType lookup table
    CONSTRAINT FK_Account_AccountType FOREIGN KEY (AccountTypeID)
        REFERENCES dbo.AccountType(AccountTypeID)
);

CREATE TABLE dbo.CustomerAccount (
    CustomerID INT NOT NULL,
    AccountID INT NOT NULL,
    DateLinked DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),

    CONSTRAINT PK_CustomerAccount PRIMARY KEY (CustomerID, AccountID),

    CONSTRAINT FK_CA_Customer FOREIGN KEY (CustomerID)
        REFERENCES dbo.Customer(CustomerID),

    CONSTRAINT FK_CA_Account FOREIGN KEY (AccountID)
        REFERENCES dbo.Account(AccountID)
);

```

```

CREATE TABLE dbo.[Transaction] (
    TransactionID    INT IDENTITY(1,1) NOT NULL,
    AccountID        INT                NOT NULL,
    CustomerID        INT                NOT NULL,
    Amount            DECIMAL(18,2)     NOT NULL,
    TransactionType    NVARCHAR(50)     NOT NULL,
    TransactionDate    DATETIME2        NOT NULL DEFAULT GETDATE(),
    DueDate            DATE              NULL,    -- For loan/credit payments only
    CompletionDate     DATETIME2        NULL,    -- NULL if transaction is still pending
    Description        NVARCHAR(255)    NULL,

    CONSTRAINT PK_Transaction PRIMARY KEY (TransactionID),

    CONSTRAINT CHK_Transaction_Type CHECK (
        TransactionType IN ('Deposit', 'Withdrawal', 'Payment', 'Transfer', 'Fee')
    ),

    -- Ensure amount is positive
    CONSTRAINT CHK_Transaction_Amount CHECK (Amount > 0),

    CONSTRAINT FK_Transaction_Account FOREIGN KEY (AccountID)
        REFERENCES dbo.Account(AccountID),

    CONSTRAINT FK_Transaction_Customer FOREIGN KEY (CustomerID)
        REFERENCES dbo.Customer(CustomerID)
);

CREATE TABLE dbo.OverdueFee (
    OverdueFeeID      INT IDENTITY(1,1) NOT NULL,
    TransactionID      INT                NOT NULL,
    FeeAmount          DECIMAL(18,2)     NOT NULL,
    DaysOverdue        INT                NOT NULL,
    FeeDate            DATETIME2        NOT NULL DEFAULT GETDATE(),
    TotalOwed          DECIMAL(18,2)     NOT NULL,
    TotalRepaid        DECIMAL(18,2)     NOT NULL DEFAULT 0.00,
    OutstandingBalance AS (TotalOwed - TotalRepaid),

    CONSTRAINT PK_OverdueFee PRIMARY KEY (OverdueFeeID),

    -- Fee amount must be positive
    CONSTRAINT CHK_OverdueFee_Amount CHECK (FeeAmount > 0),

    -- Days overdue must be at least 1
    CONSTRAINT CHK_OverdueFee_Days CHECK (DaysOverdue >= 1),

    -- TotalRepaid cannot exceed TotalOwed
    CONSTRAINT CHK_OverdueFee_Repaid CHECK (TotalRepaid <= TotalOwed),

    CONSTRAINT FK_OverdueFee_Transaction FOREIGN KEY (TransactionID)
        REFERENCES dbo.[Transaction](TransactionID)
);

```

```

CREATE TABLE dbo.Repayment (
    RepaymentID INT IDENTITY(1,1) NOT NULL,
    OverdueFeeID INT NOT NULL,
    RepaymentDate DATETIME2 NOT NULL DEFAULT GETDATE(),
    Amount DECIMAL(18,2) NOT NULL,
    PaymentMethod NVARCHAR(20) NOT NULL,

    CONSTRAINT PK_Repayment PRIMARY KEY (RepaymentID),

    -- Amount must be positive
    CONSTRAINT CHK_Repayment_Amount CHECK (Amount > 0),

    -- Restrict to the three payment methods in the brief
    CONSTRAINT CHK_Repayment_Method CHECK (
        PaymentMethod IN ('Bank Transfer', 'Card', 'Cash')
    ),

    CONSTRAINT FK_Repayment_OverdueFee FOREIGN KEY (OverdueFeeID)
        REFERENCES dbo.OverdueFee(OverdueFeeID)
);

-- Customers are frequently searched by name
CREATE NONCLUSTERED INDEX IX_Customer_LastName
    ON dbo.Customer (LastName);

-- Accounts are frequently filtered by status
CREATE NONCLUSTERED INDEX IX_Account_Status
    ON dbo.Account (Status);

-- Transactions are frequently filtered by date ranges
CREATE NONCLUSTERED INDEX IX_Transaction_Date
    ON dbo.[Transaction] (TransactionDate);

-- Transactions are frequently filtered by DueDate
-- (e.g., finding payments due within 5 days)
CREATE NONCLUSTERED INDEX IX_Transaction_DueDate
    ON dbo.[Transaction] (DueDate)
    WHERE DueDate IS NOT NULL;

CREATE UNIQUE NONCLUSTERED INDEX UQ_Account_RefNumber
    ON dbo.Account (ReferenceNumber)
    WHERE ReferenceNumber IS NOT NULL;

CREATE UNIQUE NONCLUSTERED INDEX UQ_Customer_Email
    ON dbo.Customer (Email)
    WHERE Email IS NOT NULL;

```

Messages

Commands completed successfully.

Completion time: 2026-04-15T18:07:48.9137690+01:00

70 % ✓ No issues found

✓ Query executed successfully.

OnlineBankDB

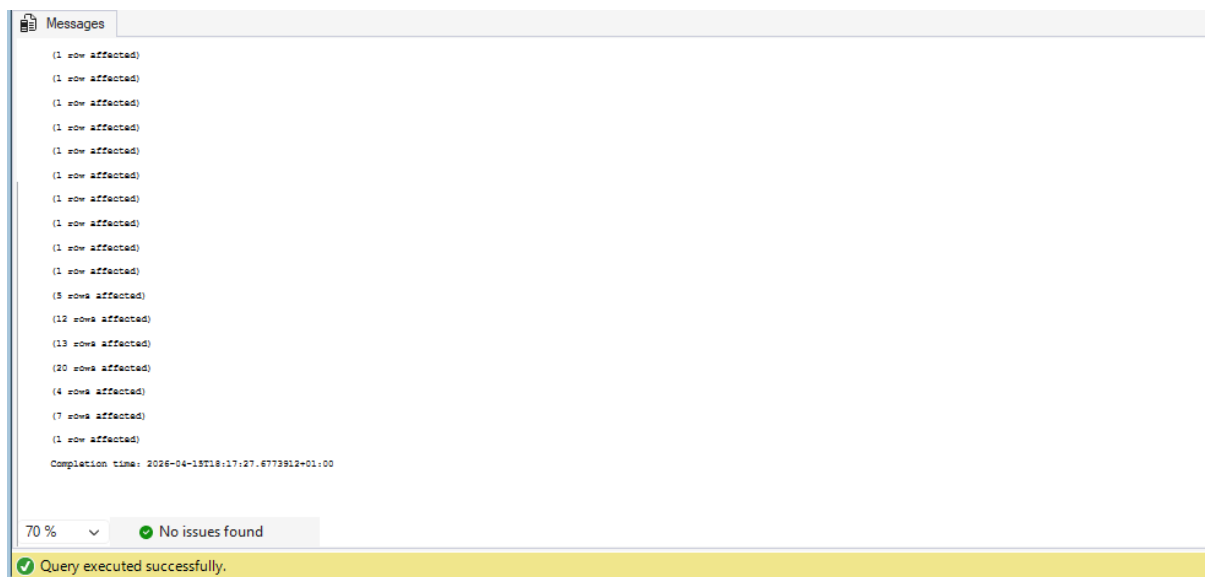
- Database Diagrams
- Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.Account
 - Columns
 - Keys
 - Constraints
 - CHK_Account_Status
 - DF_Account_Balance_5BE2A6F2
 - DF_Account_Opening_59FA5E80
 - DF_Account_Status_5AEE82B9
 - Triggers
 - Indexes
 - PK_Account (Clustered)
 - Statistics
 - dbo.AccountType
 - dbo.Address
 - dbo.ArchivedCustomer
 - dbo.Customer
 - Columns
 - Keys
 - Constraints
 - CHK_Customer_DOB
 - CHK_Customer_Email
 - DF_Customer_IsActi_4F7CD00D
 - DF_Customer_Regist_4E88ABD4
 - Triggers
 - Indexes
 - PK_Customer (Clustered)
 - UQ_Customer_Username (Unique, N
 - Statistics
 - dbo.CustomerAccount
 - dbo.OverdueFee
 - dbo.Repayment
 - dbo.Transaction
 - Columns
 - Keys
 - Constraints
 - CHK_Transaction_Amount
 - CHK_Transaction_Type
 - DF_Transacti_Trans_656C112C
 - Triggers

1.5 Populating the Database

The tables were populated with carefully designed test data. The data was not randomly generated but instead constructed to ensure that every subsequent query, stored procedure, trigger, and view could be meaningfully tested as suggested in the brief. Key scenarios built into the test data include:

A joint account shared by two customers (James and Sarah Harrison on Account 2), demonstrating the many-to-many relationship via the CustomerAccount junction table. Pending loan and credit card payments with due dates set relative to the current system date, ensuring the 'payments due in less than 5 days' function always returns valid results when executed. Overdue fees with varying repayment levels: Michael at 80% repaid (should pass the 50% threshold), Thomas at 33% and 0% on two fees, and Priya at 33% (both should fail the threshold). A closed account (Robert Taylor) with IsActive set to 0, retained for marketing purposes as specified in the brief. A frozen account (David's Emergency Checking) to demonstrate all four status values.

Password data is inserted using the HASHBYTES function with SHA2_512 and a unique salt generated via NEWID() for each customer, following the technique from the database security workshop.



-- Check row counts for all tables

```
SELECT 'Address' AS TableName, COUNT(*) AS Row_Count FROM dbo.Address
UNION ALL
SELECT 'Customer', COUNT(*) FROM dbo.Customer
UNION ALL
SELECT 'AccountType', COUNT(*) FROM dbo.AccountType
UNION ALL
SELECT 'Account', COUNT(*) FROM dbo.Account
UNION ALL
SELECT 'CustomerAccount', COUNT(*) FROM dbo.CustomerAccount
UNION ALL
SELECT 'Transaction', COUNT(*) FROM dbo.[Transaction]
UNION ALL
SELECT 'OverdueFee', COUNT(*) FROM dbo.OverdueFee
UNION ALL
SELECT 'Repayment', COUNT(*) FROM dbo.Repayment;
```

Results			Messages	
	TableName	Row_Count		
1	Address	10		
2	Customer	10		
3	AccountT...	5		
4	Account	12		
5	Customer...	13		
6	Transaction	20		
7	OverdueF...	4		
8	Repayment	7		

✓ Query executed successfully.


```

-- Verify joint account exists (should return 2 rows for Account 2)
SELECT ca.AccountID, c.FirstName, c.LastName
FROM dbo.CustomerAccount ca
INNER JOIN dbo.Customer c ON ca.CustomerID = c.CustomerID
WHERE ca.AccountID = 2;

-- Verify pending payments due within 5 days exist
SELECT t.TransactionID, a.AccountName, t.Amount, t.DueDate, t.CompletionDate
FROM dbo.[Transaction] t
INNER JOIN dbo.Account a ON t.AccountID = a.AccountID
WHERE t.CompletionDate IS NULL
AND t.DueDate IS NOT NULL
AND DATEDIFF(DAY, GETDATE(), t.DueDate) BETWEEN 0 AND 5;

-- Verify overdue fee repayment percentages
SELECT
    ovf.OverdueFeeID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    ovf.TotalOwed,
    ovf.TotalRepaid,
    ovf.OutstandingBalance,
    CAST(ROUND((ovf.TotalRepaid / ovf.TotalOwed) * 100, 1) AS DECIMAL(5,1)) AS RepaidPercentage,
    CASE WHEN ovf.TotalRepaid < (ovf.TotalOwed * 0.5) THEN 'BELOW 50%' ELSE 'ABOVE 50%' END AS Status
FROM dbo.OverdueFee ovf
INNER JOIN dbo.[Transaction] t ON ovf.TransactionID = t.TransactionID
INNER JOIN dbo.Customer c ON t.CustomerID = c.CustomerID;

```

Results		Messages	
	AccountID	FirstName	LastName
1	2	James	Harrison
2	2	Sarah	Harrison

	TransactionID	AccountName	Amount	DueDate	CompletionDate
1	9	Fatima Home Loan	500.00	2026-04-18	NULL
2	13	Michael Rewards Credit Card	150.00	2026-04-17	NULL
3	16	Thomas Personal Loan	650.00	2026-04-19	NULL

	OverdueFeeID	CustomerName	TotalOwed	TotalRepaid	OutstandingBalance	RepaidPercentage	Status
1	1	Michael OBrien	75.00	60.00	15.00	80.0	ABOVE 50%
2	2	Thomas Walker	45.50	15.50	30.00	34.1	BELOW 50%
3	3	Priya Patel	91.00	30.00	61.00	33.0	BELOW 50%
4	4	Thomas Walker	30.00	0.00	30.00	0.0	BELOW 50%

Part 2: Database Objects and Queries

2.1 Searching any Accounts by Name

A stored procedure named `uspSearchAccountsByName` was created to allow the bank to search for accounts or products by name. The procedure accepts a single parameter `@SearchName` and uses the `LIKE` operator with wildcards on both sides for flexible partial matching. Results are sorted by `OpeningDate` in descending order, so the most recently opened accounts appear first, as required by the brief.

```

GO
CREATE PROCEDURE uspSearchAccountsByName
    @SearchName NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        a.AccountID,
        a.AccountName,
        at.TypeName AS AccountType,
        a.ReferenceNumber,
        a.OpeningDate,
        a.Status,
        a.Balance
    FROM dbo.Account a
    INNER JOIN dbo.AccountType at ON a.AccountTypeID = at.AccountTypeID
    WHERE a.AccountName LIKE '%' + @SearchName + '%'
    ORDER BY a.OpeningDate DESC;
END;
GO

```

The procedure was tested with several scenarios: searching for 'Savings' (returning four accounts), 'Loan' (returning two), 'Credit' (returning two), a non-existent name 'Mortgage' (returning zero rows), and a specific account name 'Emma Student' (returning one row).

```

-- Should return 4 accounts
EXEC uspSearchAccountsByName @SearchName = 'Savings';

-- Should return 2 accounts
EXEC uspSearchAccountsByName @SearchName = 'Loan';

-- Should return 2 accounts
EXEC uspSearchAccountsByName @SearchName = 'Credit';

-- Should return 0 rows because Mortgage is not among AccountTypes
EXEC uspSearchAccountsByName @SearchName = 'Mortgage';

-- Should return 1 row
EXEC uspSearchAccountsByName @SearchName = 'Emma Student';

```

Results		Messages						
	AccountID	AccountName	AccountType	ReferenceNumber	OpeningDate	Status	Balance	
1	7	Priya Fixed Savings	Savings	NULL	2023-07-05	Active	8500.00	
2	3	Amina Premium Savings	Savings	NULL	2022-03-10	Active	12000.00	
3	1	James Everyday Savings	Savings	NULL	2022-01-15	Active	5200.00	
4	10	Robert Standard Savings	Savings	NULL	2021-06-01	Closed	0.00	
	AccountID	AccountName	AccountType	ReferenceNumber	OpeningDate	Status	Balance	
1	8	Thomas Personal Loan	Loan	LN-2023-01102	2023-10-01	Active	7800.00	
2	5	Fatima Home Loan	Loan	LN-2023-00451	2023-02-01	Active	500.00	
	AccountID	AccountName	AccountType	ReferenceNumber	OpeningDate	Status	Balance	
1	12	Priya Credit Card	Credit Card	CC-2024-00330	2024-03-15	Active	2200.00	
2	6	Michael Rewards Credit Card	Credit Card	CC-2023-00812	2023-04-20	Active	150.00	

Results		Messages						
	AccountID	AccountName	AccountType	ReferenceNumber	OpeningDate	Status	Balance	
	AccountID	AccountName	AccountType	ReferenceNumber	OpeningDate	Status	Balance	
1	9	Emma Student Checking	Checking	NULL	2024-01-15	Active	920.50	

2.2 Payments Which Are Due in Less Than 5 Days

This was implemented as a table-valued function (TVF) named `PaymentsDueSoon` rather than a stored procedure. The rationale for choosing a function over a procedure is that a TVF returns a result set which can be used directly in `SELECT` statements, `JOINS`, and `WHERE` clauses, giving the bank greater flexibility in how they consume the data. For example, the bank could write `SELECT * FROM PaymentsDueSoon() WHERE AccountType = 'Loan'` to filter the results further.

The function filters transactions where `CompletionDate` IS NULL (pending), `DueDate` is within the next five days (using `BETWEEN` with `CAST(GETDATE()) AS DATE` and `DATEADD`), and the account type is either `Loan` or `Credit Card`. The `DATEDIFF` function is used to calculate the number of days until each payment is due.

```

CREATE FUNCTION dbo.PaymentsDueSoon()
RETURNS TABLE
AS
RETURN
(
    SELECT
        t.TransactionID,
        c.CustomerID,
        c.FirstName + ' ' + c.LastName AS CustomerName,
        a.AccountID,
        a.AccountName,
        at.TypeName AS AccountType,
        t.Amount,
        t.DueDate,
        DATEDIFF(DAY, CAST(GETDATE() AS DATE), t.DueDate) AS DaysUntilDue,
        t.Description
    FROM dbo.[Transaction] t
    INNER JOIN dbo.Account a ON t.AccountID = a.AccountID
    INNER JOIN dbo.AccountType at ON a.AccountTypeID = at.AccountTypeID
    INNER JOIN dbo.Customer c ON t.CustomerID = c.CustomerID
    WHERE t.CompletionDate IS NULL -- Only pending payments
    AND t.DueDate IS NOT NULL -- Only payments with a due date
    AND t.DueDate BETWEEN CAST(GETDATE() AS DATE) -- From today
    AND DATEADD(DAY, 5, CAST(GETDATE() AS DATE)) -- To 5 days from now
    AND at.TypeName IN ('Loan', 'Credit Card') -- Only loan or credit card
);
GO

-- Should return 3 rows
-- (Fatima due in 3 days, Michael due in 2 days, Thomas due in 4 days)
SELECT * FROM dbo.PaymentsDueSoon();

-- Filter the result further by account type
SELECT * FROM dbo.PaymentsDueSoon() WHERE AccountType = 'Loan';

-- This is the case even when future payments (>5+ days) are involved
-- This query should return rows NOT in the function result
SELECT t.TransactionID, t.DueDate,
    DATEDIFF(DAY, CAST(GETDATE() AS DATE), t.DueDate) AS DaysUntilDue
FROM dbo.[Transaction] t
WHERE t.CompletionDate IS NULL AND t.DueDate IS NOT NULL
    AND DATEDIFF(DAY, CAST(GETDATE() AS DATE), t.DueDate) > 5;

```

The results for these test cases were:

Results

Messages

	TransactionID	CustomerID	CustomerName	AccountID	AccountName	AccountType	Amount	DueDate	DaysUntilDue	Description
1	9	5	Fatima Ali	5	Fatima Home Loan	Loan	500.00	2026-04-18	3	Loan instalment 3 - pending
2	13	6	Michael OBrien	6	Michael Rewards Credit Card	Credit Card	150.00	2026-04-17	2	CC FINAL payment - pending
3	16	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	2026-04-19	4	Loan payment pending

	TransactionID	CustomerID	CustomerName	AccountID	AccountName	AccountType	Amount	DueDate	DaysUntilDue	Description
1	9	5	Fatima Ali	5	Fatima Home Loan	Loan	500.00	2026-04-18	3	Loan instalment 3 - pending
2	16	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	2026-04-19	4	Loan payment pending

	TransactionID	DueDate	DaysUntilDue
1	10	2026-04-29	14
2	19	2026-04-25	10
3	20	2026-04-30	15

Query executed successfully.

2.3 Inserting New Customers

The stored procedure `uspInsertNewCustomer` handles the insertion of a new bank customer. It accepts all required customer fields including the plain-text password, which is never stored directly. Instead, the procedure generates a unique salt using `NEWID()` and hashes the password with `HASHBYTES('SHA2_512', @Password + CAST(@Salt AS NVARCHAR(36)))` before inserting the hash and salt into the Customer table.

The procedure also intelligently handles addresses: it first checks whether the address already exists in the Address table (matching on `AddressLine1` and `Postcode`). If it does, the existing `AddressID` is reused; if not, a new address record is created. This prevents duplicate address records while maintaining referential integrity.

ACID compliance is ensured through `BEGIN TRANSACTION` with `TRY...CATCH` error handling. If any part of the insertion fails (for example, a duplicate username), the entire transaction is rolled back and no partial data is committed. This follows the pattern established in the Week 9 workshop on transaction.

```

CREATE PROCEDURE uspInsertNewCustomer
    @FirstName NVARCHAR(50),
    @LastName NVARCHAR(50),
    @AddressLine1 NVARCHAR(100),
    @AddressLine2 NVARCHAR(100) = NULL,
    @City NVARCHAR(50),
    @Postcode NVARCHAR(10),
    @DateOfBirth DATE,
    @Username NVARCHAR(50),
    @Password NVARCHAR(50),
    @Email NVARCHAR(100) = NULL,
    @Telephone NVARCHAR(20) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION
    BEGIN TRY
        -- Generate a unique salt for password hashing
        DECLARE @Salt UNIQUEIDENTIFIER = NEWID();
        DECLARE @PasswordHash BINARY(64) = HASHBYTES('SHA2_512',
            @Password + CAST(@Salt AS NVARCHAR(36)));

        -- Check if the address already exists
        DECLARE @AddressID INT;
        SELECT @AddressID = AddressID
        FROM dbo.Address
        WHERE AddressLine1 = @AddressLine1 AND Postcode = @Postcode;

        -- If address doesn't exist, insert a new one
        IF @AddressID IS NULL
        BEGIN
            INSERT INTO dbo.Address (AddressLine1, AddressLine2, City, Postcode)
            VALUES (@AddressLine1, @AddressLine2, @City, @Postcode);

            SET @AddressID = SCOPE_IDENTITY();
        END

        -- Insert the customer record
        INSERT INTO dbo.Customer (FirstName, LastName, AddressID, DateOfBirth,
            Username, PasswordHash, Salt, Email, Telephone)
        VALUES (@FirstName, @LastName, @AddressID, @DateOfBirth,
            @Username, @PasswordHash, @Salt, @Email, @Telephone);

        -- Return the new customer details (excluding password hash)
        DECLARE @NewCustomerID INT = SCOPE_IDENTITY();
        SELECT @NewCustomerID AS NewCustomerID, @FirstName AS FirstName,
            @LastName AS LastName, @Username AS Username;

        COMMIT TRANSACTION
    END TRY
    BEGIN CATCH
        -- If any error occurs, roll back the entire transaction
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        -- Re-raise the error with details
        DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
        DECLARE @ErrSeverity INT = ERROR_SEVERITY();
        RAISERROR(@ErrMsg, @ErrSeverity, 1);
    END CATCH
END;
GO

```

Test cases:

-- Inserting a new customer with a new address

EXEC uspInsertNewCustomer

```
@FirstName = 'Liam',
@LastName = 'Murphy',
@AddressLine1 = '22 Victoria Road',
@AddressLine2 = NULL,
@City = 'Leeds',
@Postcode = 'LS1 5QR',
@DateOfBirth = '1993-07-14',
@Username = 'lmurphy',
@Password = 'L1amSecure!',
@Email = 'liam.murphy@email.com',
@Telephone = '07700 200001';
```

-- Verify the customer was inserted

SELECT * FROM dbo.Customer WHERE Username = 'lmurphy';

Results

Messages

	CustomerID	FirstName	LastName	AddressID	DateOfBirth	Username	PasswordHash	Salt	Email	Telephone	RegistrationDate	IsActive
1	11	Liam	Murphy	11	1993-07-14	lmurphy	0x61A8E9C89902748DC8DBA59E5514A529629F27ACE0183...	01EA3F37-1DD3-4BF1-AF2B-D455B04CF555	liam.murphy@email.com	07700 200001	2026-04-16 15:18:37.4933333	1

Results

Messages

	CustomerID	FirstName	LastName	AddressID	DateOfBirth	Username	PasswordHash	Salt	Email	Telephone	RegistrationDate	IsActive
1	1	James	Harrison	1	1985-03-15	jhartson	0x33E098E862A954FE77D9F447A03A38274A49539F98BC36...	54C1540F-B09B-475F-B98B-41A08FE21E57	james.harrison@email.com	07700 100001	2022-01-10 00:00:00.0000000	1
2	2	Sarah	Harrison	1	1987-07-22	sahartson	0x18C5273635AE93843D0A16CC83E4A5D05E668F565BF44E...	EA079EB8-EC2C-48C7-8563-F23CE59083CE	sarah.harrison@email.com	07700 100002	2022-01-10 00:00:00.0000000	1
3	3	Anina	Begum	2	1990-11-08	abegum	0xBD0ED1E48F1ECFEF1597F3C8DF776F22400C45D0C728C8...	49F7FAFD-0001-4829-9008-967076EF888A	anina.begum@email.com	07700 100003	2022-03-05 00:00:00.0000000	1
4	4	David	Chen	3	1978-05-30	dchen	0xCD95F5988E588FE8BE702FD77578927F9F63CDD0E349D...	C130FCA9-D004-47EC-A6CF-C1EC32D8CB06	david.chen@email.com	07700 100003	2022-03-05 00:00:00.0000000	1
5	5	Fatma	Ali	4	1995-09-12	fall	0xA82051B721A93A3C5FDEFC32A77F0A7559E40B5A05D8C94...	E58837C1-9A44-411F-9926-72A501F8C2A3	NULL	07700 100005	2023-01-20 00:00:00.0000000	1
6	6	Michael	Obrien	5	1982-12-01	mobrien	0xBD99244A2227C122B8980B92A5E55AE691E044188173097A...	4DE84EC2-1895-4BFF-9CAE-000BADCED170	michael.obrien@email.com	07700 100006	2023-04-11 00:00:00.0000000	1
7	7	Pritya	Patel	6	1992-02-28	ppatel	0x58B89962F3C07807915F39E407C76F15E3B3E2FC6CE3C0F...	C1443A19-8368-4758-B736-1BEA68B2F632	pritya.patel@email.com	07700 100007	2023-06-30 00:00:00.0000000	1
8	8	Thomas	Walker	7	1970-08-19	twalker	0xE2DAA64D68230EF7843113D00408F4394CF4AB9A9C1183...	2281D2F6-3792-4033-B2DC-1DF4008268D2	thomas.walker@email.com	07700 100008	2023-09-15 00:00:00.0000000	1
9	9	Emma	Jones	8	1988-04-05	ejones	0xF72A14E85E9FAF17182C4AESC6090E256163F353CACCS2...	64670252-E45D-4D02-9CB6-7D1A8C5268D2	emma.jones@email.com	07700 100009	2024-01-08 00:00:00.0000000	1
10	10	Robert	Taylor	9	1975-06-25	rtaylor	0x889BC31AD4BC68B7EB20B9F2AF54D4EA9C22522F9181...	9643C89E-1A9C-4348-96A1-BF94ED3463CA	robert.taylor@email.com	07700 100010	2021-05-20 00:00:00.0000000	0
11	11	Liam	Murphy	11	1993-07-14	lmurphy	0x61A8E9C89902748DC8DBA59E5514A529629F27ACE0183...	01EA3F37-1DD3-4BF1-AF2B-D455B04CF555	liam.murphy@email.com	07700 200001	2026-04-16 15:18:37.4933333	1

-- Inserting a customer at an existing address (14 Oak Avenue)

-- This should reuse AddressID 1 instead of creating a duplicate

EXEC uspInsertNewCustomer

```
@FirstName = 'Olivia',
@LastName = 'Harrison',
@AddressLine1 = '14 Oak Avenue',
@AddressLine2 = 'Flat 2',
@City = 'Manchester',
@Postcode = 'M1 4BT',
@DateOfBirth = '2000-11-30',
@Username = 'oharrison',
@Password = '011vi@2024',
@Email = 'olivia.harrison@email.com',
@Telephone = NULL;
```

-- Verify the customer was inserted with AddressID = 1

SELECT CustomerID, FirstName, LastName, AddressID, Username
FROM dbo.Customer WHERE Username = 'oharrison';

Results Messages

	CustomerID	FirstName	LastName	AddressID	Username
1	12	Olivia	Harrison	1	oharrison

Results		Messages											
CustomerID	FirstName	LastName	AddressID	DateOfBirth	Username	PasswordHash	Salt	Email	Telephone	RegistrationDate	IsActive		
1	James	Harrison	1	1995-03-15	jharrison	0x33E098E862A954FE7709F447A03A93B274A49539F98BC36...	54C1540F-6098-475F-88B9-41A08FE21E57	james.harrison@email.com	07700 100001	2022-01-10 00:00:00.0000000	1		
2	Sarah	Harrison	1	1987-07-22	sharrison	0x18C5273635AE39843D0A16CC83E4455D0E6685F5659F4AE...	E6079EB9-EC2C-48C7-65B3-F23CE69083CE	sarah.harrison@email.com	07700 100002	2022-01-10 00:00:00.0000000	1		
3	Amina	Begum	2	1990-11-08	abegum	0x80DEED1E48F1ECFEF1597F3C2DF7765F2240CA5DC728C8...	49F7FAFD-D001-4829-9D08-967076EF988A	amina.begum@email.com	07700 100003	2022-03-05 00:00:00.0000000	1		
4	David	Chen	3	1978-05-30	dchen	0xCD99F59885E88F6FE88E702DF77578327F9F63CDD0E349D...	C130FCA9-D0C4-47EC-A6CF-C1EC32D8C8D6	david.chen@email.com	NULL	2022-06-18 00:00:00.0000000	1		
5	Fatma	Ali	4	1995-09-12	fall	0xA82051B721A93A3CFDEFC52A77F0A7559E40B5A0D08C94...	E5B837C1-9444-411F-9926-72A501FBC2A3	NULL	07700 100005	2023-01-20 00:00:00.0000000	1		
6	Michael	O'Brien	5	1982-12-01	mbrien	0xBD99244A2227C122B8880832A5E55AE591ED44118173097A...	4DE84EC2-1B95-4BFF-9CAE-000BA0CE0170	michael.obrien@email.com	07700 100006	2023-04-11 00:00:00.0000000	1		
7	Priya	Patel	6	1992-02-28	ppatel	0x85B8E8962F3C07807915F39E407C76F15E3B3E2FC6CE3C0F...	C1443A19-B368-4758-8736-1BEA68B2F632	priya.patel@email.com	07700 100007	2023-06-30 00:00:00.0000000	1		
8	Thomas	Walker	7	1970-08-19	twalker	0xE2DAA646D68230EF7843113D00408F439ACF4A89A9C1183...	2281D2F6-3792-4033-82DC-1DF4008268D2	thomas.walker@email.com	07700 100008	2023-09-15 00:00:00.0000000	1		
9	Emma	Jones	8	1998-04-05	ejones	0xF72A14E89E9FAF17182C4AESC65090E256163F9353CACC52...	6467D252-E45D-4DD2-9CB6-7D1A8C52B324	emma.jones@email.com	07700 100009	2024-01-08 00:00:00.0000000	1		
10	Robert	Taylor	9	1975-06-25	rtaylor	0x989BC31AD4BC66BF7EB2089F2AF54D4AE9C22522F91B1...	9643C89E-1A9C-434B-96A1-8F94ED3463CA	robert.taylor@email.com	07700 100010	2021-05-20 00:00:00.0000000	0		
11	Liam	Murphy	11	1993-07-14	lmurphy	0x61A8E9C89902748DC8DBA59E5514A529629F27ACE01836...	01EA3F37-1DD3-4BF1-AF2B-D455BDAFC555	liam.murphy@email.com	07700 200001	2026-04-16 15:18:37.4933333	1		
12	Olivia	Harrison	1	2000-11-30	oharrison	0x3324E7FA6BAF3B71BD1DE67713A6C308CBDF7106C8019D...	F39E6697-0AE2-4F69-9535-B2A6A18EB978	olivia.harrison@email.com	NULL	2026-04-16 15:22:55.1866667	1		

-- Inserting a customer with a DUPLICATE username — should FAIL

EXEC uspInsertNewCustomer

```

@FirstName = 'Micah',
@LastName = 'Reki',
@AddressLine1 = '19 Reiner Street',
@AddressLine2 = NULL,
@City = 'Sheffield',
@Postcode = 'S9 1DT',
@DateOfBirth = '1990-01-01',
@Username = 'lmurphy', -- This username already exists (Liam Murphy uses it).
@Password = 'MicP@sh19',
@Email = 'micahreki@email.com',
@Telephone = NULL;

```

Messages

Msg 50000, Level 14, State 1, Procedure uspInsertNewCustomer, Line 59 [Batch Start Line 1091]
Violation of UNIQUE KEY constraint 'UQ_Customer_Username'. Cannot insert duplicate key in object 'dbo.Customer'. The duplicate key value is (lmurphy).

Completion time: 2026-04-16T15:29:04.2502293+01:00

-- Inserting a customer with optional fields left NULL

EXEC uspInsertNewCustomer

```

@FirstName = 'Noah',
@LastName = 'Williams',
@AddressLine1 = '5 Bridge Street',
@AddressLine2 = NULL,
@City = 'Chester',
@Postcode = 'CH1 1RE',
@DateOfBirth = '1988-03-22',
@Username = 'nwilliams',
@Password = 'N0@hW!lls',
@Email = NULL,
@Telephone = NULL;

```

Results		Messages											
NewCustomerID	FirstName	LastName	Username										
14	Noah	Williams	nwilliams										

Results	Messages	CustomerID	FirstName	LastName	AddressID	DateOfBirth	Username	PasswordHash	Salt	Email	Telephone	RegistrationDate	IsActive
1		1	James	Harrison	1	1985-03-15	jhartson	0x33E098E8862A54FE77D9F447A03A53B274A49539F98BC36...	54C1540F-8098-475F-8888-41A08FE21E57	james.hartson@email.com	07700 100001	2022-01-10 00:00:00.0000000	1
2		2	Sarah	Harrison	1	1987-07-22	sharrison	0x1BC5273635AE93843D0A16CC83E4A55D5E6685F565BF44E...	EA079EB8-EC2C-48C7-B5B3-F23CE59083CE	sarah.hartson@email.com	07700 100002	2022-01-10 00:00:00.0000000	1
3		3	Amina	Begum	2	1990-11-08	abegum	0xBDDED1E48F1ECFEF1597F3C8DF7765F22400A5DC728C8...	49F7AFD-0001-4829-9D08-967076EF888A	amina.begum@email.com	07700 100003	2022-03-05 00:00:00.0000000	1
4		4	David	Chen	3	1978-05-30	dchen	0xCD95F59885E8BF6FE8BE702FD775789275F63CDD3E349D...	C130FCA9-00C4-47EC-A6CF-C1EC92D8CB06	david.chen@email.com	NULL	2022-06-18 00:00:00.0000000	1
5		5	Fatima	Ali	4	1995-09-12	fall	0xA92051B721A93A3CFDEFC2A7770A7599E40B5A05D8C94...	E58837C1-9A44-411F-9926-72A501FBC2A3	NULL	07700 100005	2023-01-20 00:00:00.0000000	1
6		6	Michael	Obrien	5	1982-12-01	mobrien	0xBD99244A2227C122B8980892A5E55AE691E041188173097A...	4DE84EC2-1895-48FF-9CAE-000BA0CE0170	michael.obrien@email.com	07700 100006	2023-04-11 00:00:00.0000000	1
7		7	Priya	Patel	6	1992-02-28	ppatel	0x85BE89962F3C07B07915F39E407C76F15E3B3E2FC6C3C0F...	C1443A19-8368-4758-8736-1BEA6B82F632	priya.patel@email.com	07700 100007	2023-06-30 00:00:00.0000000	1
8		8	Thomas	Walker	7	1970-08-19	twalker	0xE2DAA646D6B230EF7843113D00A08F439ACF4A89AC1183...	2281D2F6-3792-4033-B2DC-1DF400826BD2	thomas.walker@email.com	07700 100008	2023-09-15 00:00:00.0000000	1
9		9	Emma	Jones	8	1998-04-05	ejones	0x772A14E85E9FAF17182C4E9CE6090E25616F3935C4CC52...	64670252-E450-4D02-9CB6-7D1A8C528324	emma.jones@email.com	07700 100009	2024-01-08 00:00:00.0000000	1
10		10	Robert	Taylor	9	1975-06-25	rtaylor	0x989BC91AD4BC6E6BF7EB2089F2AF54D4AEAS9C22522F91B1...	9643C89E-1A9C-434B-96A1-BF94ED3463CA	robert.taylor@email.com	07700 100010	2021-05-20 00:00:00.0000000	0
11		11	Liam	Murphy	11	1993-07-14	lmurphy	0x61A8E9C89902748DC8DBA59E5514A529629F27ACE01836...	01EA3F37-1DD3-48F1-AF2B-D455ED4CF555	liam.murphy@email.com	07700 200001	2026-04-16 15:18:37.4933333	1
12		12	Olivia	Harrison	1	2000-11-30	oharrison	0x3324E7FA68AF3B71BD1DE67713A6C308CB0F7106C8019D...	F35E6697-0AE2-4F69-9535-B2A6A18EB978	olivia.hartson@email.com	NULL	2026-04-16 15:22:55.1866667	1
13		14	Noah	Williams	13	1988-03-22	nwilliams	0x59A0C7CED2EB1D2C82185851062669C86EC180AC10FE699...	C4D97E76-5042-4B38-A9C9-DEB72A7EC39A	NULL	NULL	2026-04-16 15:30:47.3866667	1

2.4 Updating an Existing Customer

The stored procedure `uspUpdateCustomer` allows modification of customer details that is already existing. It is designed so that only the fields provided as parameters are updated; any parameter left as `NULL` retains the existing value using the `ISNULL` function. This prevents accidental overwriting of data when the bank only needs to change a single field such as a telephone number.

The procedure first validates that the specified `CustomerID` exists, raising an appropriate error if it does not. For address changes, it follows the same pattern as the insert procedure, checking for existing addresses before creating new records. Like the insert procedure, it uses `BEGIN TRANSACTION` with `TRY...CATCH` for ACID compliance.

```

CREATE PROCEDURE uspUpdateCustomer
    @CustomerID INT,
    @FirstName NVARCHAR(50) = NULL,
    @LastName NVARCHAR(50) = NULL,
    @AddressLine1 NVARCHAR(100) = NULL,
    @AddressLine2 NVARCHAR(100) = NULL,
    @City NVARCHAR(50) = NULL,
    @Postcode NVARCHAR(10) = NULL,
    @Email NVARCHAR(100) = NULL,
    @Telephone NVARCHAR(20) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION
    BEGIN TRY
        -- Check customer exists
        IF NOT EXISTS (SELECT 1 FROM dbo.Customer WHERE CustomerID = @CustomerID)
        BEGIN
            RAISERROR('Customer with ID %d does not exist.', 16, 1, @CustomerID);
            RETURN;
        END

        -- If address fields are provided, handle address update
        IF @AddressLine1 IS NOT NULL AND @Postcode IS NOT NULL
        BEGIN
            DECLARE @NewAddressID INT;

            -- Check if the new address already exists
            SELECT @NewAddressID = AddressID
            FROM dbo.Address
            WHERE AddressLine1 = @AddressLine1 AND Postcode = @Postcode;

            -- If not, create it
            IF @NewAddressID IS NULL
            BEGIN
                INSERT INTO dbo.Address (AddressLine1, AddressLine2, City, Postcode)
                VALUES (@AddressLine1, @AddressLine2, @City, @Postcode);
            END
        END
    END TRY
    CATCH
    BEGIN
        ROLLBACK TRANSACTION;
    END
END

```

```

        SET @NewAddressID = SCOPE_IDENTITY();
    END

    -- Update the customer's AddressID
    UPDATE dbo.Customer SET AddressID = @NewAddressID
    WHERE CustomerID = @CustomerID;
END

-- Update other customer fields (only if provided)
UPDATE dbo.Customer
SET
    FirstName = ISNULL(@FirstName, FirstName),
    LastName = ISNULL(@LastName, LastName),
    Email = ISNULL(@Email, Email),
    Telephone = ISNULL(@Telephone, Telephone)
WHERE CustomerID = @CustomerID;

-- Return the updated customer record
SELECT c.CustomerID, c.FirstName, c.LastName,
       a.AddressLine1, a.AddressLine2, a.City, a.Postcode,
       c.Email, c.Telephone
FROM dbo.Customer c
INNER JOIN [Address] a ON c.AddressID = a.AddressID
WHERE c.CustomerID = @CustomerID;

COMMIT TRANSACTION
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrSeverity INT = ERROR_SEVERITY();
    RAISERROR(@ErrMsg, @ErrSeverity, 1);
END CATCH
END;
GO

```

Test cases:

```

-- Update only the email for Customer 5 (Fatima Ali)
-- BEFORE state:
SELECT CustomerID, FirstName, LastName, Email, Telephone
FROM dbo.Customer WHERE CustomerID = 5;

EXEC uspUpdateCustomer
    @CustomerID = 5,
    @Email = 'fatima.ali@email.com';

-- AFTER state (only email should change):
SELECT CustomerID, FirstName, LastName, Email, Telephone
FROM dbo.Customer WHERE CustomerID = 5;

```

Results		Messages			
	CustomerID	FirstName	LastName	Email	Telephone
1	5	Fatima	Ali	NULL	07700 100005

Before

	CustomerID	FirstName	LastName	Email	Telephone
1	5	Fatima	Ali	fatima.ali@email.com	07700 100005

After

-- Update name and telephone for Customer 9 (Emma Jones)

EXEC uspUpdateCustomer

@CustomerID = 9,

@FirstName = 'Emily',

@LastName = 'Jones-Smith',

@Telephone = '07700 999999';

SELECT CustomerID, FirstName, LastName, Telephone
FROM dbo.Customer WHERE CustomerID = 9;

Results		Messages										
	CustomerID	FirstName	LastName	AddressID	DateOfBirth	Username	PasswordHash	Sal	Email	Telephone	RegistrationDate	IsActive
1	1	James	Harrison	1	1985-03-15	jharrison	0x31E98E8962A954FE77D9F447A03A3B274A4953F9B8C36...	54C1540F-8098-475F-8688-41A08FE21E57	james.harrison@email.com	07700 100001	2022-01-10 00:00:00.0000000	1
2	2	Sarah	Harrison	1	1987-07-22	s.harrison	0x18C5273635AE93843D0A16CC3E4A5D5E6689F568F44E...	EAD79E8B-EC2C-48C7-85B3-F23CE6983CE	sarah.harrison@email.com	07700 100002	2022-01-10 00:00:00.0000000	1
3	3	Amina	Begum	2	1990-11-08	abegum	0xBDD0ED1E48F1ECFEF1597F3C80F7765F2240CA5DC728C8...	49F7FAD-D001-4829-9008-967076FE888A	amina.begum@email.com	07700 100003	2022-03-05 00:00:00.0000000	1
4	4	David	Chen	3	1978-05-30	dchen	0xC095F59885E8BFEBE702FD77578327F9F63C0DE3490...	C130FCA9-D0C4-47EC-A6CF-C1EC3D8CB06	david.chen@email.com	NULL	2022-06-18 00:00:00.0000000	1
5	5	Fatima	Ali	4	1995-09-12	fall	0xA82051B721A93A3CFD0FC52A77F0A7598E4085A0D9C94...	E5B837C1-9A44-411F-9926-72A501FBC2A3	NULL	07700 100005	2023-01-20 00:00:00.0000000	1
6	6	Michael	O'Brien	5	1982-12-01	mobrien	0xBD99244A227C122B89808932A5E55AE691ED4118173097A...	4DE84EC2-1895-4BFF-9CAE-00B8A0CE0170	michael.obrien@email.com	07700 100006	2023-04-11 00:00:00.0000000	1
7	7	Priya	Patel	6	1992-02-28	ppatel	0x85B89962F3C07B07915F39E407C76F15E3B3E2FC6CE3C0F...	C1443A19-8368-4758-8736-1BEA6B82F632	priya.patel@email.com	07700 100007	2023-06-30 00:00:00.0000000	1
8	8	Thomas	Walker	7	1970-08-19	twalker	0xE2DA4646D68230EF7843113D00408F439ACF4A89AC1183...	2281D2F6-3792-4033-82DC-10F400826BD2	thomas.walker@email.com	07700 100008	2023-09-15 00:00:00.0000000	1
9	9	Emma	Jones	8	1998-04-05	ejones	0xF72A14E85E9FAF17182C4A5EC6090E2B6163F9353AC52...	64670252-E45D-4D02-9CB6-7D1A8C52B324	emma.jones@email.com	07700 100009	2024-01-08 00:00:00.0000000	1
10	10	Robert	Taylor	9	1975-06-25	rtaylor	0x889BC91AD4BCCE6BF7EB20B9F2AF54D4AE9C2252F91B1...	9643C89E-1A5C-4348-96A1-BF94ED3463CA	robert.taylor@email.com	07700 100010	2021-05-20 00:00:00.0000000	0
11	11	Liam	Murphy	11	1993-07-14	lmurphy	0x61A8E9C89902748DC8D8A59E5514A529629F27ACE01836...	01EA3F37-1DD3-4BF1-AF2B-D4558D4CF555	liam.murphy@email.com	07700 200001	2026-04-16 15:18:37.4933333	1
12	12	Olivia	Harrison	1	2000-11-30	oharrison	0x3324E7FA6BAF3B71BD1DE67713A6C308CBDF7106C8019D...	F35E6697-0AE2-4F69-9535-B2A6A18EB978	olivia.harrison@email.com	NULL	2026-04-16 15:22:55.1866667	1
13	14	Noah	Williams	13	1988-03-22	nwilliams	0x59A0C7CED2EB1D2C8218585106266C86EC180AC10FE699...	C4D97E76-5042-4B38-A9C9-DEB72A7E2C9A	NULL	NULL	2026-04-16 15:30:47.3866667	1

Before

Results

Messages

	CustomerID	FirstName	LastName	AddressLine1	AddressLine2	City	Postcode	Email	Telephone
1	9	Emily	Jones-Smith	19 Ash Grove	Floor 1	Wigan	WN1 1YB	emma.jones@email.com	07700 999999

Emily Jones changed to Emily Jones-Smith.

-- Update address (move to a new address)

EXEC uspUpdateCustomer

@CustomerID = 3,

@AddressLine1 = '15 New Road',

@City = 'Manchester',

@Postcode = 'M2 4WU';

SELECT c.CustomerID, c.FirstName, a.AddressLine1, a.City, a.Postcode
FROM dbo.Customer c INNER JOIN dbo.Address a ON c.AddressID = a.AddressID
WHERE c.CustomerID = 3;

Results		Messages			
	CustomerID	FirstName	AddressLine1	City	Postcode
1	3	Amina	27 Elm Street	Salford	M5 3AQ

Before

Results		Messages			
	CustomerID	FirstName	AddressLine1	City	Postcode
1	3	Amina	15 New Road	Manchester	M2 4WU

After

-- Updating a customer that doesn't exist — should raise error

EXEC uspUpdateCustomer

@CustomerID = 29,

@FirstName = 'Megan';

Messages	
Msg 50000, Level 16, State 1, Procedure uspUpdateCustomer, Line 72 [Batch Start Line 1158] Customer with ID 29 does not exist.	
Completion time: 2026-04-16T15:54:28.2292491+01:00	

2.5 Snapshot View of Transactions Having Overdue Fees

A view named TransactionsWithOverdueFees was created to give the bank a consolidated view of all transactions along with any associated overdue fees. The view joins five tables: Transaction, Account, AccountType, Customer, and OverdueFee.

A LEFT JOIN is used on the OverdueFee table rather than an INNER JOIN, so that transactions without overdue fees are still included in the results. The view also includes a computed TransactionStatus column using a CASE expression, which classifies each transaction as 'Pending', 'Completed Late', or 'Completed On Time' based on the CompletionDate and DueDate values.

The benefits of using a view here are significant. It simplifies what would otherwise be a complex multi-table join into a single queryable object. It can also be used as a security mechanism, allowing users to query the view without requiring direct access to the underlying tables. Since only the view definition is stored and not the data itself, it introduces no storage overhead (Ramakrishnan and Gehrke, 2003).

```

CREATE VIEW dbo.TransactionsWithOverdueFees
AS
SELECT
    t.TransactionID,
    c.CustomerID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    a.AccountID,
    a.AccountName,
    at.TypeName AS AccountType,
    t.Amount AS TransactionAmount,
    t.TransactionType,
    t.TransactionDate,
    t.DueDate,
    t.CompletionDate,
    CASE
        WHEN t.CompletionDate IS NULL THEN 'Pending'
        WHEN t.DueDate IS NOT NULL AND t.CompletionDate > t.DueDate THEN 'Completed Late'
        ELSE 'Completed On Time'
    END AS TransactionStatus,
    ovf.OverdueFeeID,
    ovf.FeeAmount,
    ovf.DaysOverdue,
    ovf.FeeDate,
    ovf.TotalOwed,
    ovf.TotalRepaid,
    ovf.OutstandingBalance
FROM dbo.[Transaction] t
INNER JOIN dbo.Account a ON t.AccountID = a.AccountID
INNER JOIN dbo.AccountType at ON a.AccountTypeID = at.AccountTypeID
INNER JOIN dbo.Customer c ON t.CustomerID = c.CustomerID
LEFT JOIN dbo.OverdueFee ovf ON t.TransactionID = ovf.TransactionID;
GO

```

Test cases:

-- Q3 TEST CASES

-- Test 1: View all rows

```

SELECT * FROM dbo.TransactionsWithOverdueFees
ORDER BY TransactionDate DESC;

```

Results Messages																	
	TransactionID	CustomerID	CustomerName	AccountID	AccountName	AccountType	TransactionAmount	TransactionType	TransactionDate	DueDate	CompletionDate	TransactionStatus	OverdueFeeID	FeeAmount	DaysOverdue	FeeDate	TotalOwed
1	10	5	Fatima Ali	5	Fatima Home Loan	Loan	500.00	Payment	2026-04-14 18:17:27.6366667	2026-04-29	NULL	Pending	NULL	NULL	NULL	NULL	NULL
2	13	6	Michael Obrien	6	Michael Rewards Credit Card	Credit Card	150.00	Payment	2026-04-14 18:17:27.6366667	2026-04-17	NULL	Pending	NULL	NULL	NULL	NULL	NULL
3	16	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	Payment	2026-04-14 18:17:27.6366667	2026-04-19	NULL	Pending	NULL	NULL	NULL	NULL	NULL
4	19	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	Payment	2026-04-14 18:17:27.6366667	2026-04-25	NULL	Pending	NULL	NULL	NULL	NULL	NULL
5	20	7	Priya Patel	12	Priya Credit Card	Credit Card	400.00	Payment	2026-04-14 18:17:27.6366667	2026-04-30	NULL	Pending	NULL	NULL	NULL	NULL	NULL
6	9	5	Fatima Ali	5	Fatima Home Loan	Loan	500.00	Payment	2026-04-13 18:17:27.6366667	2026-04-18	NULL	Pending	NULL	NULL	NULL	NULL	NULL
7	18	7	Priya Patel	12	Priya Credit Card	Credit Card	400.00	Payment	2024-06-01 00:00:00.0000000	2024-06-15	2024-06-28 0...	Completed Late	3	91.00	13	2024-0...	91.00
8	15	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	Payment	2024-04-01 00:00:00.0000000	2024-04-15	2024-04-22 0...	Completed Late	2	45.50	7	2024-0...	45.50
9	15	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	Payment	2024-04-01 00:00:00.0000000	2024-04-15	2024-04-22 0...	Completed Late	4	30.00	7	2024-0...	30.00
10	17	9	Emily Jones-S...	9	Emma Student Checking	Checking	300.00	Deposit	2024-02-01 00:00:00.0000000	NULL	2024-02-01 0...	Completed On T...	NULL	NULL	NULL	NULL	NULL
11	14	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	Payment	2024-01-01 00:00:00.0000000	2024-01-15	2024-01-12 0...	Completed On T...	NULL	NULL	NULL	NULL	NULL
12	12	6	Michael Obrien	6	Michael Rewards Credit Card	Credit Card	250.00	Payment	2023-11-01 00:00:00.0000000	2023-11-15	2023-11-25 0...	Completed Late	1	75.00	10	2023-1...	75.00
13	8	5	Fatima Ali	5	Fatima Home Loan	Loan	500.00	Payment	2023-09-01 00:00:00.0000000	2023-09-15	2023-09-12 0...	Completed On T...	NULL	NULL	NULL	NULL	NULL
14	11	6	Michael Obrien	6	Michael Rewards Credit Card	Credit Card	300.00	Payment	2023-08-01 00:00:00.0000000	2023-08-15	2023-08-10 0...	Completed On T...	NULL	NULL	NULL	NULL	NULL
15	7	5	Fatima Ali	5	Fatima Home Loan	Loan	500.00	Payment	2023-06-01 00:00:00.0000000	2023-06-15	2023-06-10 0...	Completed On T...	NULL	NULL	NULL	NULL	NULL
16	6	4	David Chen	4	David Growth Investment	Investment	10000.00	Deposit	2023-06-01 00:00:00.0000000	NULL	2023-06-01 0...	Completed On T...	NULL	NULL	NULL	NULL	NULL
17	2	1	James Hartson	1	James Everyday Savings	Savings	500.00	Withdrawal	2022-06-15 00:00:00.0000000	NULL	2022-06-15 0...	Completed On T...	NULL	NULL	NULL	NULL	NULL
4																	
Query executed successfully																	
										localhost (17.0 RTM) MIC47ZFR 0.000000 OnlineBackEnd 00:00:00 Row: 1 Col: 1 21 rows							

Query executed successfully.

localhost (17.0 RTM) | MICHELLE_R_UBRIGH (80) | OnlineBankDB | 00:00:00 | Row: 1, Col: 1 | 21 rows

-- Filter to only show transactions with overdue fees

-- Should return 4 rows

```

SELECT * FROM dbo.TransactionsWithOverdueFees
WHERE OverdueFeeID IS NOT NULL
ORDER BY TransactionDate;

```

	TransactionID	CustomerID	CustomerName	AccountID	AccountName	Account Type	TransactionAmount	TransactionType	TransactionDate	DueDate	CompletionDate	TransactionStatus	OverdueFeeID	FeeAmount	DaysOverdue	FeeDate
1	12	6	Michael OBrien	6	Michael Rewards Credit Card	Credit Card	250.00	Payment	2023-11-01 00:00:00.0000000	2023-11-15	2023-11-25 00:00:00.0000000	Completed Late	1	75.00	10	2023-11-25
2	15	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	Payment	2024-04-01 00:00:00.0000000	2024-04-15	2024-04-22 00:00:00.0000000	Completed Late	2	45.50	7	2024-04-22
3	15	8	Thomas Walker	8	Thomas Personal Loan	Loan	650.00	Payment	2024-04-01 00:00:00.0000000	2024-04-15	2024-04-22 00:00:00.0000000	Completed Late	4	30.00	7	2024-04-22
4	18	7	Priya Patel	12	Priya Credit Card	Credit Card	400.00	Payment	2024-06-01 00:00:00.0000000	2024-06-15	2024-06-28 00:00:00.0000000	Completed Late	3	91.00	13	2024-06-28

-- Show only pending transactions

```
SELECT TransactionID, CustomerName, AccountName, TransactionAmount,
       DueDate, TransactionStatus
FROM dbo.TransactionsWithOverdueFees
WHERE TransactionStatus = 'Pending';
```

	TransactionID	CustomerName	AccountName	TransactionAmount	DueDate	TransactionStatus
1	9	Fatima Ali	Fatima Home Loan	500.00	2026-04-18	Pending
2	10	Fatima Ali	Fatima Home Loan	500.00	2026-04-29	Pending
3	13	Michael OBrien	Michael Rewards Credit Card	150.00	2026-04-17	Pending
4	16	Thomas Walker	Thomas Personal Loan	650.00	2026-04-19	Pending
5	19	Thomas Walker	Thomas Personal Loan	650.00	2026-04-25	Pending
6	20	Priya Patel	Priya Credit Card	400.00	2026-04-30	Pending

-- Show late payments with their overdue fee details

```
SELECT CustomerName, AccountName, TransactionAmount,
       DueDate, CompletionDate, DaysOverdue, FeeAmount,
       TotalOwed, TotalRepaid, OutstandingBalance
FROM dbo.TransactionsWithOverdueFees
WHERE TransactionStatus = 'Completed Late';
```

	CustomerName	AccountName	TransactionAmount	DueDate	CompletionDate	DaysOverdue	FeeAmount	TotalOwed	TotalRepaid	OutstandingBalance
1	Michael OBrien	Michael Rewards Credit Card	250.00	2023-11-15	2023-11-25 00:00:00.0000000	10	75.00	75.00	60.00	15.00
2	Thomas Walker	Thomas Personal Loan	650.00	2024-04-15	2024-04-22 00:00:00.0000000	7	45.50	45.50	15.50	30.00
3	Thomas Walker	Thomas Personal Loan	650.00	2024-04-15	2024-04-22 00:00:00.0000000	7	30.00	30.00	0.00	30.00
4	Priya Patel	Priya Credit Card	400.00	2024-06-15	2024-06-28 00:00:00.0000000	13	91.00	91.00	30.00	61.00

2.6 Trigger for Closing an Account Automatically

A trigger named `t_AutoCloseAccount` was created on the `Transaction` table. It fires after an `UPDATE` operation and implements the following business rule: when the final scheduled payment on a Loan or Credit Card account is marked as completed, the account status is changed from 'Active' to 'Closed', the date of closure is recorded, and then the account balance is set to zero.

The trigger works by checking three conditions when a transaction's `CompletionDate` is updated from `NULL` to a non-`NULL` value: (1) the associated account is of type `Loan` or `Credit Card`, (2) the account is currently `Active`, and (3) there are no remaining pending payments (transactions with `CompletionDate` IS `NULL` and a non-`NULL` `DueDate`) for that account. Only when every one of these conditions are true does the trigger change the status on the account.

The trigger was tested in two stages for Fatima's Home Loan (Account 5). First, Transaction 9 was marked as completed. Since Transaction 10 was still pending, the trigger correctly did not close the account. Then Transaction 10 (the final payment) was completed, and the trigger successfully changed the account status to 'Closed'

and recorded the closure date. A second test was performed on Michael's Credit Card (Account 6), confirming the same behaviour.

```
CREATE TRIGGER t_AutoCloseAccount
ON dbo.[Transaction]
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Only proceed if CompletionDate was updated from NULL to a value
    IF UPDATE(CompletionDate)
    BEGIN
        DECLARE @AccountID INT;
        DECLARE @AccountTypeID INT;
        DECLARE @AccountTypeName NVARCHAR(50);
        DECLARE @PendingCount INT;

        -- Get the account details from the updated transaction
        SELECT @AccountID = i.AccountID
        FROM inserted i
        INNER JOIN deleted d ON i.TransactionID = d.TransactionID
        WHERE d.CompletionDate IS NULL AND i.CompletionDate IS NOT NULL;

        -- If no relevant update, exit
        IF @AccountID IS NULL RETURN;

        -- Check if the account is a Loan or Credit Card and currently Active
        SELECT @AccountTypeID = a.AccountTypeID,
               @AccountTypeName = at.TypeName
        FROM dbo.Account a
        INNER JOIN dbo.AccountType at ON a.AccountTypeID = at.AccountTypeID
        WHERE a.AccountID = @AccountID
              AND at.TypeName IN ('Loan', 'Credit Card')
              AND a.Status = 'Active';

        -- If not a Loan/CC or not Active, exit
        IF @AccountTypeName IS NULL RETURN;

        -- Count remaining pending payments for this account
        SELECT @PendingCount = COUNT(*)
        FROM dbo.[Transaction]
        WHERE AccountID = @AccountID
              AND CompletionDate IS NULL
              AND DueDate IS NOT NULL;
```

```

-- If no more pending payments, close the account
IF @PendingCount = 0
BEGIN
    UPDATE dbo.Account
    SET Status = 'Closed',
        StatusDate = GETDATE(),
        ClosureDate = CAST(GETDATE() AS DATE),
        Balance = 0.00
    WHERE AccountID = @AccountID;

    PRINT 'TRIGGER: Account ' + CAST(@AccountID AS VARCHAR(10))
        + ' (' + @AccountTypeName + ') has been automatically closed.'
        + ' All scheduled payments have been completed.';
END
END
END;
GO

```

Test cases:

```

-- =====
-- Q4 TEST CASES
-- =====

-- TRIGGER TEST CASE 1: Fatima's Home Loan (Account 5)
-- Currently has 2 pending payments (TransactionIDs 9 and 10)

-- BEFORE state:
SELECT AccountID, AccountName, Status, Balance FROM Account WHERE AccountID = 5;
SELECT TransactionID, CompletionDate, Description
FROM dbo.[Transaction] WHERE AccountID = 5 AND CompletionDate IS NULL;

```

Results

Messages

	AccountID	AccountName	Status	Balance
1	5	Fatima Home Loan	Active	500.00

	TransactionID	CompletionDate	Description
1	9	NULL	Loan instalment 3 - pending
2	10	NULL	Loan FINAL instalment - pending

```

-- Complete transaction 9 (not the last one)
-- The trigger should not close the account because transaction 10 is still pending
UPDATE dbo.[Transaction]
SET CompletionDate = GETDATE()
WHERE TransactionID = 9;

-- Account should still be Active
SELECT AccountID, AccountName, Status, Balance FROM dbo.Account WHERE AccountID = 5;

```

Results

Messages

	AccountID	AccountName	Status	Balance
1	5	Fatima Home Loan	Active	500.00

-- Complete transaction 10 (final payment)

-- The trigger should close the account now

UPDATE dbo.[Transaction]

SET CompletionDate = GETDATE()

WHERE TransactionID = 10;

-- AFTER state: Account should now be Closed

SELECT AccountID, AccountName, Status, StatusDate, ClosureDate, Balance

FROM dbo.Account WHERE AccountID = 5;

Messages	
TRIGGER: Account 5 (Loan) has been automatically closed. All scheduled payments have been completed.	
(1 row affected)	
Completion time: 2026-04-16T16:31:35.4515567+01:00	

Results

Messages

	AccountID	AccountName	Status	StatusDate	ClosureDate	Balance
1	5	Fatima Home Loan	Closed	2026-04-16 16:31:35.4400000	2026-04-16	0.00

-- TRIGGER TEST CASE 2: Michael's Credit Card (Account 6)

-- Has 1 pending payment (TransactionID 13)

-- BEFORE state:

SELECT AccountID, AccountName, Status, Balance FROM dbo.Account WHERE AccountID = 6;

Results

Messages

	AccountID	AccountName	Status	Balance
1	6	Michael Rewards Credit Card	Active	150.00

-- Complete the final credit card payment

UPDATE dbo.[Transaction]

SET CompletionDate = GETDATE()

WHERE TransactionID = 13;

-- AFTER state: Account should now be Closed

SELECT AccountID, AccountName, Status, StatusDate, ClosureDate, Balance

FROM dbo.Account WHERE AccountID = 6;

Messages	
TRIGGER: Account 6 (Credit Card) has been automatically closed. All scheduled payments have been completed.	
(1 row affected)	
Completion time: 2026-04-16T16:35:04.8326537+01:00	

Results

Messages

	AccountID	AccountName	Status	StatusDate	ClosureDate	Balance
1	6	Michael Rewards Credit Card	Closed	2026-04-16 16:35:04.8233333	2026-04-16	0.00

2.7 Customers Having Less Than 50% Overdue Fees Repayment

A SELECT query was written to identify customers who have repaid less than 50% of their overdue fees. The query filters at the individual fee level using the condition

WHERE TotalRepaid < (TotalOwed * 0.5). The query results are then grouped by customer and the number of matching fees are counted.

The query uses NULLIF to prevent division-by-zero errors when calculating the overall repaid percentage. It also uses CAST and ROUND for clean decimal output.

The expected results, based on the test data, are: Thomas Walker with 2 under-repaid fees (Fee 2 at 34% repaid and Fee 4 at 0% repaid), and Priya Patel with 1 under-repaid fee (Fee 3 at 33% repaid). Michael O'Brien does not appear because his single fee (Fee 1) is 80% repaid, which exceeds the 50% threshold.

```
SELECT
    c.CustomerID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    COUNT(ovf.OverdueFeeID) AS NumberOfUnderRepaidFees,
    SUM(ovf.TotalOwed) AS TotalAmountOwed,
    SUM(ovf.TotalRepaid) AS TotalAmountRepaid,
    CAST(ROUND(SUM(ovf.TotalRepaid) / NULLIF(SUM(ovf.TotalOwed), 0) * 100, 1)
        AS DECIMAL(5,1)) AS OverallRepaidPercentage
FROM dbo.OverdueFee ovf
INNER JOIN dbo.[Transaction] t ON ovf.TransactionID = t.TransactionID
INNER JOIN dbo.Customer c ON t.CustomerID = c.CustomerID
WHERE ovf.TotalRepaid < (ovf.TotalOwed * 0.5)
GROUP BY c.CustomerID, c.FirstName, c.LastName
ORDER BY NumberOfUnderRepaidFees DESC;
```

GO

	CustomerID	CustomerName	NumberOfUnderRepaidFees	TotalAmountOwed	TotalAmountRepaid	OverallRepaidPercentage
1	8	Thomas Walker	2	75.50	15.50	20.5
2	7	Priya Patel	1	91.00	30.00	33.0

2.8 Test Cases of Database Objects

All database objects were tested using a structured approach. For each object, the test cases demonstrate both expected successful outcomes and deliberate failure scenarios to prove that error handling works correctly. The test strategy includes:

For stored procedures: execution with valid parameters, execution with edge-case values, and execution with invalid data that should trigger the CATCH block. For the trigger: a two-step test showing the trigger does not fire prematurely (when pending payments remain) and fires correctly on the final payment. For the view: queries with different filters to demonstrate all three transaction statuses. For the function: both positive results and verification that payments due beyond 5 days are excluded.

All test case screenshots are included in the relevant sections above, showing both the T-SQL execution and the result sets.

2.9 Some Additional Database Objects

7a: Customer Account Summary Procedure

A stored procedure named `uspCustomerAccountSummary` was created to provide a comprehensive overview for any customer. It returns three result sets: the customer's personal details and address, a summary of all their accounts with transaction counts and aggregate figures (total deposits, total withdrawals, pending payments), and any outstanding overdue fees. This would be valuable for customer service representatives handling enquiries, providing a complete picture in a single call.

```
CREATE PROCEDURE uspCustomerAccountSummary
    @CustomerID INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Check customer exists
    IF NOT EXISTS (SELECT 1 FROM dbo.Customer WHERE CustomerID = @CustomerID)
    BEGIN
        RAISERROR('Customer with ID %d does not exist.', 16, 1, @CustomerID);
        RETURN;
    END

    -- Customer details
    SELECT c.CustomerID,
           c.FirstName + ' ' + c.LastName AS FullName,
           c.Email, c.Telephone, c.RegistrationDate, c.IsActive,
           a.AddressLine1, a.City, a.Postcode
    FROM dbo.Customer c
    INNER JOIN [Address] a ON c.AddressID = a.AddressID
    WHERE c.CustomerID = @CustomerID;

    -- Account summary with transaction counts
    SELECT
        acc.AccountID,
        acc.AccountName,
        at.TypeName AS AccountType,
        acc.Status,
        acc.Balance,
        acc.OpeningDate,
        COUNT(t.TransactionID) AS TotalTransactions,
        ISNULL(SUM(CASE WHEN t.TransactionType = 'Deposit' THEN t.Amount ELSE 0 END), 0) AS TotalDeposits,
        ISNULL(SUM(CASE WHEN t.TransactionType = 'Withdrawal' THEN t.Amount ELSE 0 END), 0) AS TotalWithdrawals,
        ISNULL(SUM(CASE WHEN t.CompletionDate IS NULL AND t.DueDate IS NOT NULL THEN 1 ELSE 0 END), 0) AS PendingPayments
    FROM dbo.CustomerAccount ca
    INNER JOIN dbo.Account acc ON ca.AccountID = acc.AccountID
    INNER JOIN dbo.AccountType at ON acc.AccountTypeID = at.AccountTypeID
    LEFT JOIN dbo.[Transaction] t ON acc.AccountID = t.AccountID
    WHERE ca.CustomerID = @CustomerID
    GROUP BY acc.AccountID, acc.AccountName, at.TypeName,
             acc.Status, acc.Balance, acc.OpeningDate;

    -- Outstanding overdue fees for this customer
    SELECT
        ovf.OverdueFeeID,
        acc.AccountName,
        ovf.TotalOwed,
        ovf.TotalRepaid,
        ovf.OutstandingBalance,
        ovf.DaysOverdue,
        ovf.FeeDate
    FROM dbo.OverdueFee ovf
    INNER JOIN dbo.[Transaction] t ON ovf.TransactionID = t.TransactionID
    INNER JOIN dbo.Account acc ON t.AccountID = acc.AccountID
    WHERE t.CustomerID = @CustomerID
           AND ovf.OutstandingBalance > 0;
END;
GO
```

Test cases:

-- Q7a TEST CASES

-- Customer 8 (Thomas) has a loan account with two overdue fees

EXEC uspCustomerAccountSummary @CustomerID = 8;

-- Customer 1 (James) has multiple accounts including joint

EXEC uspCustomerAccountSummary @CustomerID = 1;

Results Messages										
CustomerID	FullName	Email	Telephone	RegistrationDate	IsActive	AddressLine1	City	Postcode		
1	8	Thomas Walker	thomas.walker@email.com	07700 100008	2023-09-15 00:00:00.0000000	1	31 Pine Crescent	Stockport	SK1 1EB	
AccountID	AccountName	Account Type	Status	Balance	OpeningDate	TotalTransactions	TotalDeposits	TotalWithdrawals	PendingPayments	
1	8	Thomas Personal Loan	Loan	Active	7800.00	2023-10-01	4	0.00	0.00	2
OverdueFeeID	AccountName	TotalOwed	TotalRepaid	OutstandingBalance	DaysOverdue	FeeDate				
1	2	Thomas Personal Loan	45.50	15.50	30.00	7	2024-04-23 00:00:00.0000000			
2	4	Thomas Personal Loan	30.00	0.00	30.00	7	2024-04-23 00:00:00.0000000			
CustomerID	FullName	Email	Telephone	RegistrationDate	IsActive	AddressLine1	City	Postcode		
1	1	James Harrison	james.harrison@email.com	07700 100001	2022-01-10 00:00:00.0000000	1	14 Oak Avenue	Manchester	M1 4BT	
AccountID	AccountName	Account Type	Status	Balance	OpeningDate	TotalTransactions	TotalDeposits	TotalWithdrawals	PendingPayments	
1	1	James Everyday Savings	Savings	Active	5200.00	2022-01-15	2	2000.00	500.00	0
2	2	Harrison Joint Checking	Checking	Active	3150.75	2022-01-15	2	1500.00	200.00	0
OverdueFeeID	AccountName	TotalOwed	TotalRepaid	OutstandingBalance	DaysOverdue	FeeDate				

Thomas Walker has two late payments on his loan account overdue by 7 days each. James Harrison has multiple accounts (including one joint checking account) with no loan or credit accounts. Hence, no overdue fees exist.

7b: Customer Archival Trigger

A trigger named `t_ArchiveDeactivatedCustomer` was created to support the brief's requirement that customer information is retained for marketing purposes when an account is closed. When a customer's `IsActive` flag is updated from 1 to 0, this AFTER UPDATE trigger automatically copies the customer's key details (name, email, telephone) into an `ArchivedCustomer` table along with a timestamp.

This approach means the customer record remains in the main `Customer` table (preserving referential integrity with their transactions and accounts) while a snapshot is separately stored for marketing use. The trigger follows the archive pattern demonstrated in the Week 8 workshop.

```
CREATE TABLE dbo.ArchivedCustomer (
    ArchiveID INT IDENTITY(1,1) NOT NULL,
    CustomerID INT NOT NULL,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100) NULL,
    Telephone NVARCHAR(20) NULL,
    DeactivationDate DATETIME2 NOT NULL DEFAULT GETDATE(),

    CONSTRAINT PK_ArchivedCustomer PRIMARY KEY (ArchiveID)
);
GO
```

```

CREATE TRIGGER t_ArchiveDeactivatedCustomer
ON dbo.Customer
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Only proceed if IsActive changed from 1 to 0
    IF UPDATE(IsActive)
    BEGIN
        INSERT INTO dbo.ArchivedCustomer (CustomerID, FirstName, LastName,
            Email, Telephone)
        SELECT i.CustomerID, i.FirstName, i.LastName, i.Email, i.Telephone
        FROM inserted i
        INNER JOIN deleted d ON i.CustomerID = d.CustomerID
        WHERE d.IsActive = 1 AND i.IsActive = 0;
    END
END;
GO

```

Test cases:

-- Q7b TEST CASES

-- BEFORE: Check ArchivedCustomer table is empty

```
SELECT * FROM dbo.ArchivedCustomer;
```

Results		Messages				
ArchiveID	CustomerID	FirstName	LastName	Email	Telephone	DeactivationDate

-- Deactivate Customer 5 (Fatima Ali)

```
UPDATE dbo.Customer SET IsActive = 0 WHERE CustomerID = 5;
```

-- AFTER: Fatima should now appear in the archive

```
SELECT * FROM dbo.ArchivedCustomer;
```

Results

Messages

	ArchiveID	CustomerID	FirstName	LastName	Email	Telephone	DeactivationDate
1	1	5	Fatima	Ali	fatima.ali@email.com	07700 100005	2026-04-18 18:31:12.5566667

-- Verify Fatima is still in the Customer table (retained for marketing)

```
SELECT CustomerID, FirstName, LastName, IsActive
FROM dbo.Customer WHERE CustomerID = 5;
```

Results

Messages

	CustomerID	FirstName	LastName	IsActive
1	5	Fatima	Ali	0

Results		Messages										
	CustomerID	FirstName	LastName	AddressID	DateOfBirth	Username	PasswordHash	Salt	Email	Telephone	RegistrationDate	IsActive
1	1	James	Harrison	1	1985-03-15	jharrison	0x33E098E862A954FE77D9F447A03A38274A49539F98BC36...	54C1540F-B098-475F-B8B8-41A08FE21E57	james.harrison@email.com	07700 100001	2022-01-10 00:00:00.0000000	1
2	2	Sarah	Harrison	1	1987-07-22	sharrison	0x1BC5273635AE93843D0A16CC83E4A55D9E6685F5658F44E...	EAD79EB8-EC2C-48C7-85B3-F23CE69083CE	sarah.harrison@email.com	07700 100002	2022-01-10 00:00:00.0000000	1
3	3	Amina	Begum	14	1990-11-08	abegum	0xBDDDED1E48F1ECFEF1597F3C8DF7765F22400CA5DC728C8...	49F7AFD-D001-4829-9D08-967076EF888A	amina.begum@email.com	07700 100003	2022-03-05 00:00:00.0000000	1
4	4	David	Chen	3	1978-05-30	dchen	0xCDF95F59885E8BF6FE8BE702FD77578927F5F63CDD0E349D...	C130FCA9-D0C4-47EC-A6CF-C1EC32DBC8D6	david.chen@email.com	NULL	2022-06-18 00:00:00.0000000	1
5	5	Fatima	Ali	4	1995-09-12	fall	0xA82051B721A93A3CFDFC52A77F0A7559E40B5A05D8C94...	E58837C1-9A44-411F-9926-72A501F8C2A3	fatima.ali@email.com	07700 100005	2023-01-20 00:00:00.0000000	0
6	6	Michael	O'Brien	5	1982-12-01	mobrien	0xB0D99244A2227C122B8980892A5E55AE91E044188173097A...	4DE84EC2-1B95-4BFF-9CAE-000BA0CE0170	michael.obrien@email.com	07700 100006	2023-04-11 00:00:00.0000000	1
7	7	Priya	Patel	6	1992-02-28	ppatel	0x858E89962F3C07B07915F39E407C76F15E3B3E2FC6CE3C0F...	C1443A19-B368-4758-B736-18EA6882F632	priya.patel@email.com	07700 100007	2023-06-30 00:00:00.0000000	1
8	8	Thomas	Walker	7	1970-08-19	twalker	0xE2DAA646D6B230EF7843113D00408F439ACF4A89A9C1183...	2281D2F6-3792-4033-82DC-1DF400826BD2	thomas.walker@email.com	07700 100008	2023-09-15 00:00:00.0000000	1
9	9	Emily	Jones-Smith	8	1998-04-05	ejones	0xF72A14E85E9FAF17182C4AESCCE6090E256163F9353C4CC52...	6470252-E45D-4D02-9CB6-7D1A8C526324	emma.jones@email.com	07700 999999	2024-01-08 00:00:00.0000000	1
10	10	Robert	Taylor	9	1975-06-25	rtaylor	0x989BC91AD4BC8EBF7EB20B9F2AF54D4AE9AC2252F91B1...	9643C38E-1A9C-434B-96A1-8F94ED3463CA	robert.taylor@email.com	07700 100010	2021-05-20 00:00:00.0000000	0
11	11	Liam	Murphy	11	1993-07-14	lmurphy	0x61A8E9C899027480C8DBA59E3514A529629F2F7ACE01836...	01EA3F37-1D03-4BF1-AF2B-D4558D4CF555	liam.murphy@email.com	07700 200001	2026-04-16 15:18:37.4933333	1
12	12	Olivia	Harrison	1	2000-11-30	oharrison	0x3324E7FA68AF3B71BD1DE67713A6C308CB0F7106C8019D...	F35E6697-0AE2-4F69-9535-E2A6A1BE6978	olivia.harrison@email.com	NULL	2026-04-16 15:22:55.1866667	1
13	14	Noah	Williams	13	1988-03-22	nwilliams	0x59A0C7CED2EB1D2C82185851062668C86EC1804C10FE699...	C4D93E76-5042-4B38-A9C8-DEB72A7E2C3A	NULL	NULL	2026-04-16 15:30:47.3866667	1

Additional Recommendations

Data Integrity and Concurrency

Data integrity in this banking system is enforced at multiple levels. At the schema level, primary key, foreign key, CHECK, UNIQUE, and NOT NULL constraints prevent invalid data from being entered. The computed column for OutstandingBalance in the OverdueFee table guarantees mathematical consistency without relying on application logic.

At the procedural level, all stored procedures that modify data use BEGIN TRANSACTION with TRY...CATCH blocks. If any statement within the transaction fails, the entire operation is rolled back, preventing partial updates that could leave the database in an inconsistent state. This ensures the atomicity and consistency properties of ACID transactions (Connolly and Begg, 2015).

For concurrency, the default transaction isolation level in SQL Server is READ COMMITTED, which prevents dirty reads by ensuring that a transaction cannot read data modified by another uncommitted transaction. For this banking scenario, READ COMMITTED is appropriate for most operations. However, for critical financial operations such as balance transfers between accounts, a higher isolation level such as SERIALIZABLE could be considered to prevent phantom reads, though this comes at the cost of reduced concurrency and potential deadlocks.

An additional consideration is the use of optimistic concurrency control for the web-facing application layer. By including a ROWVERSION column in the Account table, the application can detect if another user has modified the same record between the time it was read and the time the update is submitted, preventing lost updates without holding locks.

Database Security

Security has been addressed at several layers within this design. Authentication credentials are never stored in plain text. Passwords are hashed using the SHA2_512 algorithm with a unique salt per customer, generated via NEWID(). This approach means that even if the database were compromised, the original passwords could not be directly recovered. The salting prevents attackers from using pre-computed rainbow tables to crack the hashes (Stallings, 2017).

At the database level, SQL Server's role-based access control should be used to restrict what each user or application can do. For this banking system, the following roles are recommended: a CustomerService role with SELECT and UPDATE permissions on customer-related tables but no DELETE permissions, a Finance role with SELECT permissions on all tables for reporting purposes, and an AppUser role (created WITHOUT LOGIN) for the front-end application with tightly controlled permissions. Schemas can be used as security boundaries, grouping customer-related objects under a Customers schema and granting permissions at the schema level rather than on individual tables.

SQL injection is a significant risk for any system with a web-facing front end. All stored procedures in this design use parameterised queries rather than dynamic SQL, which

is the primary defence against SQL injection attacks. The application layer should never construct SQL statements by concatenating user input.

Additionally, Transparent Data Encryption (TDE) should be enabled at the database level to encrypt the database files at rest, protecting against physical theft of storage media. For data in transit, SSL/TLS encryption should be enforced for all connections to the SQL Server instance.

Database Backup and Recovery

Given that the bank processes hundreds of transactions daily and short system downtime of a few hours is acceptable (as stated in the brief), a backup strategy using a combination of full and differential backups is recommended.

A full backup should be scheduled nightly during off-peak hours (for example, 2:00 AM). This captures the entire database state and serves as the baseline for recovery. Differential backups should be taken every four hours during business hours, capturing only the data that has changed since the last full backup. Transaction log backups should be taken every 15 to 30 minutes to enable point-in-time recovery, minimising data loss in the event of a failure.

The recovery model should be set to FULL rather than SIMPLE, as this enables point-in-time recovery from transaction log backups. With the SIMPLE recovery model, only recovery to the most recent full or differential backup is possible, which could result in the loss of several hours' worth of transactions.

Backups should be verified using the RESTORE VERIFYONLY command with the WITH CHECKSUM option to confirm they are not corrupt and can be successfully restored. A regular test restore should be performed to a separate environment at least monthly to confirm that the backup and recovery process works end to end.

For disaster recovery, backups should be stored in at least two geographically separate locations. SQL Server's backup to URL functionality can be used to store backups in cloud storage, providing off-site redundancy. A documented recovery plan, including step-by-step restore procedures and responsible personnel, should be maintained and tested periodically.

Conclusions

Microsoft SQL Server was used to implement the design of a relational database system for an online banking platform in this report. The database was normalised to Third Normal Form, with each design decision documented and justified in the context of eliminating redundancy and preventing anomalies.

The schema comprises eight tables covering all entities specified in the brief: customers, addresses, account types, accounts, transactions, overdue fees, and repayments. The inclusion of a junction table for the Customer-Account relationship provides the flexibility to support joint accounts in future, meeting the brief's requirement for a schema that can accommodate evolving client requirements.

A range of database objects were implemented, including four stored procedures and functions (for searching accounts, identifying upcoming payments, inserting new customers, and updating existing customer records), a view consolidating transactions with overdue fees, a trigger for automatically closing loan and credit card accounts upon final payment, and a query for identifying customers with under-repaid overdue fees. Two additional objects were also provided: a customer account summary procedure and an archival trigger for deactivated customers.

Throughout the implementation, data integrity was enforced through constraints, ACID-compliant transactions, and parameterised queries. Security was addressed through password hashing with unique salts, role-based access control recommendations, and defence against SQL injection. Backup and recovery recommendations were also tailored to the specific requirements of the banking scenario, including a strategy combining differential and transaction log backups alongside the FULL recovery model.

NHS Prescribing Data: SQL Querying & Analysis

Task 2 Report: NHS Prescribing Data: SQL Querying & Analysis

Introduction

This report documents the analysis of NHS prescribing data for the Bolton area using T-SQL queries in Microsoft SQL Server Management Studio (SSMS) 2022. The dataset, released by the National Health Service in England, provides detailed information on prescriptions issued by medical practices in the Bolton region. The data spans four related tables covering medical practices, prescribed drugs, individual prescriptions, and a monthly summary of prescribing activity.

The task requires creating a database, importing the four CSV files, establishing the appropriate relationships between the tables, and writing a series of analytical queries. These queries address specific business questions that a pharmaceutical company might ask when trying to understand prescribing patterns, identify commercially relevant trends, and detect data quality issues across the Bolton area.

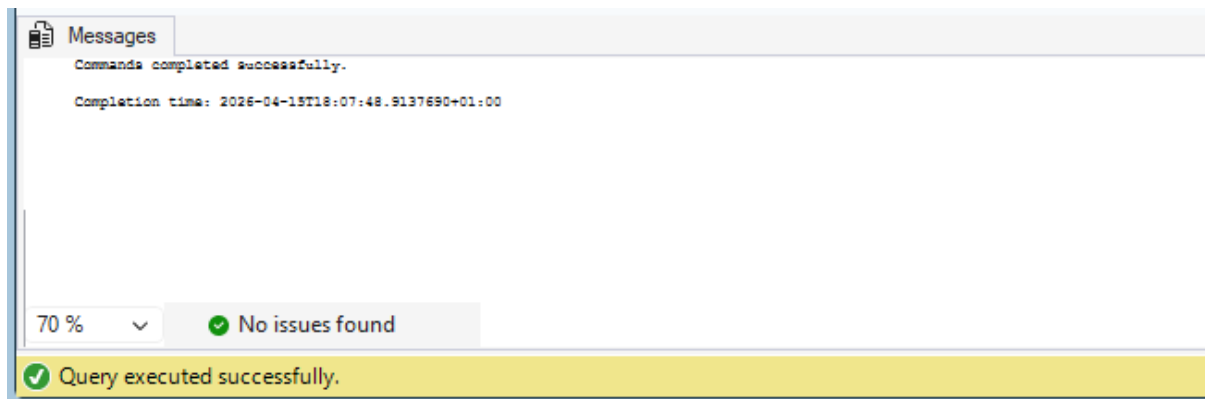
The report is structured in two parts. Part 1 covers the database creation, data import process, and the establishment of primary and foreign key constraints. Part 2 presents each query with its T-SQL code, an explanation of the approach taken, and the result set.

Part 1: Database Creation and Data Import

1.1 Creating the Database

The database was created using the CREATE DATABASE statement with the name PrescriptionsDB as specified in the brief. The USE statement was then executed to switch the active database context.

```
CREATE DATABASE PrescriptionsDB;  
GO  
USE PrescriptionsDB;  
GO
```



1.2 Importing the CSV Files

The four CSV files were imported into the database using the Import Flat File wizard in SSMS. This was accessed by right-clicking on the PrescriptionsDB database in the Object Explorer and selecting Tasks, then Import Flat File. For each file, the wizard was pointed to the relevant CSV, and the column data types were reviewed on the preview screen before completing the import.

The import was carried out in the following order, chosen to respect the foreign key dependencies that would later be added: Medical_Practice and Drugs were imported first as they have no dependencies on other tables, followed by Prescriptions (which references both Medical_Practice and Drugs), and finally Prescription_Summary (which references Medical_Practice).

Medical_Practice.csv:

Specify Input File

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)

Specify Input File

This operation will create a table from your input file.

Location of file to be imported

C:\Users\brigh\OneDrive - University of Salford\ADB\Assessment\Medical_Practice.cs [Browse...](#)

New table name:

Medical_Practice

Table schema:

dbo

[< Previous](#)
[Next >](#)
[Cancel](#)

Preview Data

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)

Preview Data

This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

PRACTICE_CODE	PRACTICE_NAME	ADDRESS_1	ADDRESS_2	ADDRESS_3	ADDRE
P82034	EDGWORTH M...	EGERTON/DUN...	DARWEN ROAD	BROMLEY CROSS	BOLTON
P82652	FARNWORTH F...	FARNWORTH F...	FREDRICK STR...	BOLTON	
P82037	FIG TREE MEDI...	FARNWORTH H...	FREDERICK STR...	FARNWORTH	BOLTON
P82633	GREAT LEVER O...	GREAT LEVER H...	RUPERT STREET...	BOLTON	LANCA'
P82022	HALLIWELL SU...	THE HALLIWEL...	LINDFIELD DRIVE	HALLIWELL	BOLTON
P82029	HALLIWELL SU...	THE HALLIWEL...	LINDFIELD DRIV...	BOLTON	LANCA'
P82626	HALLIWELL SU...	THE HALLIWEL...	LINDFIELD DRIV...	BOLTON	LANCA'
P82016	HARWOOD ME...	HARWOOD ME...	HOUGH FOLD ...	BOLTON	LANCA'
P82031	HEATON MEDI...	HEATON MEDI...	2 LUCY STREET	HEATON, BOLT...	
P82030	DEANE MEDIC...	155-157 DEANE...		BOLTON	
P82007	KEARSLEY MED...	KEARSLEY MED...	JACKSON ST, K...	BOLTON	LANCA'
Y00747	INTERGRATED ...	BREIGHTMET H...	BREIGHTMET F...	BOLTON	LANCA'
P82661	INTERMEDIATE ...	DARLEY COURT	SHEPHERD CR...	BOLTON	
P82004	SWAN LANE M...	SWAN LANE M...	SWAN LANE	BOLTON	LANCA'
P82018	THE ALASTAIR ...	BREIGHTMET H...	BREIGHTMET F...	BRIGHTMET, B...	LANCA'
P82001	THE DUNSTAN ...	BREIGHTMET H...	BREIGHTMET F...	BRIGHTMET, B...	LANCA'
P82021	THE OAKS FAM...	CROMPTON H...	501 CROMPTO...	BOLTON	

☒ Use Rich Data Type Detection - may provide a closer type fit. However, cells with anomalous values may be dropped.

[< Previous](#)
[Next >](#)
[Cancel](#)



Modify Columns

[Introduction](#)

[Specify Input File](#)

[Preview Data](#)

Modify Columns

[Summary](#)

[Results](#)

[Help](#)

Modify Columns

This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	☐ Allow Nulls	
PRACTICE_CODE	nvarchar(10)	☒	☐	
PRACTICE_NAME	nvarchar(100)	☐	☐	
ADDRESS_1	nvarchar(100)	☐	☒	
ADDRESS_2	nvarchar(100)	☐	☒	
ADDRESS_3	nvarchar(100)	☐	☒	
ADDRESS_4	nvarchar(100)	☐	☒	
POSTCODE	nvarchar(10)	☐	☒	

Row granularity of error reporting (performance impact with smaller ranges)

No Range

[< Previous](#)

[Next >](#)

[Cancel](#)



Results

[Introduction](#)

[Specify Input File](#)

[Preview Data](#)

[Modify Columns](#)

[Summary](#)

Results

[Help](#)



Operation Complete

Summary:

Name	Result
☒ Insert Data	Success

[< Previous](#)

[Next >](#)

[Close](#)

Drugs.csv:

Specify Input File

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

Help

Specify Input File

This operation will create a table from your input file.

Location of file to be imported

C:\Users\brigh\OneDrive - University of Salford\ADB\Assessment\Drugs.csv

Browse...

New table name:

Drugs

Table schema:

dbo

< Previous

Next >

Cancel

Preview Data

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

Help

Preview Data

This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

BNF_CODE	CHEMICAL_SUBS	BNF_DESCRIPTION	BNF_CHAPTER_P
0205051L0AAA...	Lisinopril	Lisinopril 5mg t...	02: Cardiovascu...
0205051L0AAA...	Lisinopril	Lisinopril 10mg...	02: Cardiovascu...
0205051L0AAA...	Lisinopril	Lisinopril 20mg...	02: Cardiovascu...
0205051M0AAA...	Perindopril erb...	Perindopril erb...	02: Cardiovascu...
0205051M0AAA...	Perindopril erb...	Perindopril erb...	02: Cardiovascu...
0205051M0AAA...	Perindopril erb...	Perindopril erb...	02: Cardiovascu...
0205051R0AAA...	Ramipril	Ramipril 1.25m...	02: Cardiovascu...
0205051R0AAA...	Ramipril	Ramipril 2.5mg ...	02: Cardiovascu...
0205051R0AAA...	Ramipril	Ramipril 5mg c...	02: Cardiovascu...
0205051R0AAA...	Ramipril	Ramipril 10mg ...	02: Cardiovascu...
0205051R0AAA...	Ramipril	Ramipril 1.25m...	02: Cardiovascu...
0205051R0AAA...	Ramipril	Ramipril 2.5mg ...	02: Cardiovascu...
0205051R0AAA...	Ramipril	Ramipril 10mg ...	02: Cardiovascu...
0205052AEAAA...	Sacubitril/valsa...	Sacubitril 24mg...	02: Cardiovascu...
0205052B0AAA...	Olmesartan me...	Olmesartan me...	02: Cardiovascu...
0205052C0AAA...	Candesartan cil...	Candesartan 2...	02: Cardiovascu...
0205052C0AAA...	Candesartan cil...	Candesartan 4...	02: Cardiovascu...
0205052C0AAA...	Candesartan cil...	Candesartan 8...	02: Cardiovascu...

☒ Use Rich Data Type Detection - may provide a closer type fit. However, cells with anomalous values may be dropped.

< Previous

Next >

Cancel


Modify Columns

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)

Modify Columns
 This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	☐ Allow Nulls	
BNF_CODE	nvarchar(20)	☒	☐	
CHEMICAL_SUBSTANCE_BNF_DESCR	nvarchar(200)	☐	☐	
BNF_DESCRIPTION	nvarchar(400)	☐	☐	
BNF_CHAPTER_PLUS_CODE	nvarchar(200)	☐	☐	

Row granularity of error reporting (performance impact with smaller ranges)

< Previous
Next >
Cancel


Results

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)


Operation Complete

Summary:

Name	Result
 Insert Data	Success

< Previous
Next >
Close

Prescriptions.csv:



Specify Input File

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

Help

Specify Input File

This operation will create a table from your input file.

Location of file to be imported

C:\Users\brigh\OneDrive - University of Salford\ADB\Assessment\Prescriptions.csv

Browse...

New table name:

Prescriptions

Table schema:

dbo

< Previous

Next >

Cancel



Preview Data

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

Help

Preview Data

This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

PREScription_C	PRACTICE_CODE	BNF_CODE	QUANTITY	ITEMS	ACTUAL
0	P82034	0205051L0AAA...	28	11	8.56974
1	P82034	0205051L0AAA...	56	4	6.18294
2	P82034	0205051L0AAA...	28	12	10.4707
3	P82034	0205051L0AAA...	84	1	2.59289
4	P82034	0205051L0AAA...	56	10	17.3272
5	P82034	0205051L0AAA...	84	2	5.52236
6	P82034	0205051L0AAA...	112	4	14.7097
7	P82034	0205051L0AAA...	56	12	22.1390
8	P82034	0205051L0AAA...	28	9	8.35794
9	P82034	0205051M0AAA...	30	1	0.91931
10	P82034	0205051M0AAA...	28	1	0.96321
11	P82034	0205051M0AAA...	56	1	1.80468
12	P82034	0205051M0AAA...	30	3	3.17866
13	P82034	0205051M0AAA...	28	2	2.95488
14	P82034	0205051M0AAA...	84	1	4.19817
15	P82034	0205051M0AAA...	30	1	1.47094
16	P82034	0205051R0AAA...	56	2	3.54025
...	...	...	...	...	...

☒ Use Rich Data Type Detection - may provide a closer type fit. However, cells with anomalous values may be dropped.

< Previous

Next >

Cancel



Modify Columns

[Introduction](#)

[Specify Input File](#)

[Preview Data](#)

Modify Columns

[Summary](#)

[Results](#)

[Help](#)

Modify Columns

This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	☐ Allow Nulls	
PRESCRIPTION_CODE	int	☒	☐	
PRACTICE_CODE	nvarchar(10)	☐	☐	
BNF_CODE	nvarchar(20)	☐	☐	
QUANTITY	float	☐	☐	
ITEMS	int	☐	☐	
ACTUAL_COST	float	☐	☐	

Row granularity of error reporting (performance impact with smaller ranges)

No Range

[< Previous](#)

[Next >](#)

[Cancel](#)



Results

[Introduction](#)

[Specify Input File](#)

[Preview Data](#)

[Modify Columns](#)

[Summary](#)

Results

[Help](#)



Operation Complete

Summary:

Name	Result
☒ Insert Data	Success

[< Previous](#)

[Next >](#)

[Close](#)

Prescription_Summary.csv:



Specify Input File

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)

Specify Input File

This operation will create a table from your input file.

Location of file to be imported

C:\Users\brigh\OneDrive - University of Salford\ADB\Assessment\Prescription_Summ
Browse...

New table name:

Prescription_Summary

Table schema:

dbo

▼

< Previous

Next >

Cancel



Preview Data

[Introduction](#)
[Specify Input File](#)
[Preview Data](#)
[Modify Columns](#)
[Summary](#)
[Results](#)

[Help](#)

Preview Data

This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

PRACTICE_CODE	TOTAL_ITEMS	TOTAL_QUANTIT	TOTAL_COST	REPORT_MONTH
P82034	11	28	8.56974	December
P82034	4	56	6.18294	April
P82034	12	28	10.47075	January
P82034	1	84	2.59289	February
P82034	10	56	17.32726	September
P82034	2	84	5.52236	May
P82034	4	112	14.70977	March
P82034	12	56	22.13905	January
P82034	9	28	8.35794	September
P82034	1	30	0.91931	October
P82034	1	28	0.96321	February
P82034	1	56	1.80468	March
P82034	3	30	3.17866	July
P82034	2	28	2.95488	December
P82034	1	84	4.19817	May
P82034	1	30	1.47094	April
P82034	2	56	3.54025	July
P82034	4	28	3.56505	May

☒ Use Rich Data Type Detection - may provide a closer type fit. However, cells with anomalous values may be dropped.

< Previous

Next >

Cancel



Modify Columns

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

[Help](#)

Modify Columns

This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	☐ Allow Nulls	
PRACTICE_CODE	nvarchar(10) ▾	☐	☐	
TOTAL_ITEMS	int ▾	☐	☐	
TOTAL_QUANTITY	float ▾	☐	☐	
TOTAL_COST	float ▾	☐	☐	
REPORT_MONTH	nvarchar(20) ▾	☐	☐	

Row granularity of error reporting (performance impact with smaller ranges)

No Range ▾

< Previous

Next >

Cancel



Results

Introduction

Specify Input File

Preview Data

Modify Columns

Summary

Results

[Help](#)

✓ Operation Complete

Summary:

Name	Result
✓ Insert Data	Success

< Previous

Next >

Close

1.3 Adding Primary and Foreign Key Constraints

After the data was imported, primary and foreign key constraints were added using ALTER TABLE statements. This approach was chosen because the Import Flat File wizard creates tables without constraints by default. Adding them after import is a valid and practical approach, as it avoids potential conflicts during the bulk import process. The constraints were added as follows.

The Medical_Practice table uses PRACTICE_CODE as its primary key, uniquely identifying each of the 60 practices. The Drugs table uses BNF_CODE as its primary key, uniquely identifying each of the 6,148 drug records. The Prescriptions table uses PRESCRIPTION_CODE as its primary key, with two foreign keys linking it to Medical_Practice (via PRACTICE_CODE) and Drugs (via BNF_CODE). The Prescription_Summary table required a surrogate primary key (SUMMARY_ID) added as an IDENTITY column, since no single column in the original dataset uniquely identifies each row. It also has a foreign key linking PRACTICE_CODE back to the Medical_Practice table.

```
ALTER TABLE dbo.Prescriptions
ADD CONSTRAINT FK_Prescriptions_Practice
FOREIGN KEY (PRACTICE_CODE)
REFERENCES dbo.Medical_Practice(PRACTICE_CODE);
```

```
ALTER TABLE dbo.Prescriptions
ADD CONSTRAINT FK_Prescriptions_Drugs
FOREIGN KEY (BNF_CODE)
REFERENCES dbo.Drugs(BNF_CODE);
```

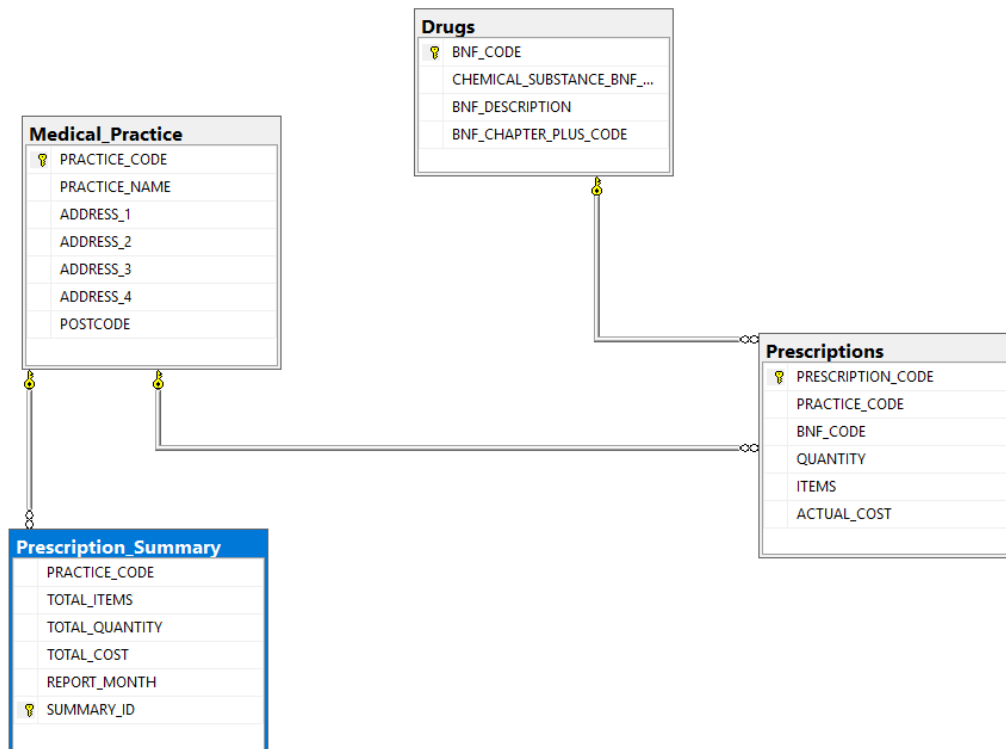
```
ALTER TABLE dbo.Prescription_Summary
ADD SUMMARY_ID INT IDENTITY(1,1) NOT NULL;
```

```
ALTER TABLE dbo.Prescription_Summary
ADD CONSTRAINT PK_Prescriptions_Summary PRIMARY KEY (SUMMARY_ID);
```

```
ALTER TABLE dbo.Prescription_Summary
ADD CONSTRAINT FK_Summary_Practice
FOREIGN KEY (PRACTICE_CODE)
REFERENCES dbo.Medical_Practice(PRACTICE_CODE);
```

1.4 Database Diagram

The diagram below shows the four tables and their relationships. Medical_Practice sits at the centre, with one-to-many relationships to both Prescriptions and Prescription_Summary via PRACTICE_CODE. The Drugs table has a one-to-many relationship with Prescriptions via BNF_CODE.



1.5 Verifying the Import

A verification query was run to confirm the row counts for each table matched the expected values from the CSV files: Medical_Practice with 60 rows, Drugs with 6,148 rows, Prescriptions with 116,815 rows, and Prescription_Summary with 116,815 rows.

```

-- Verify row counts after import
SELECT 'Medical_Practice' AS TableName, COUNT(*) AS [RowCount] FROM dbo.Medical_Practice
UNION ALL
SELECT 'Drugs', COUNT(*) FROM dbo.Drugs
UNION ALL
SELECT 'Prescriptions', COUNT(*) FROM dbo.Prescriptions
UNION ALL
SELECT 'Prescription_Summary', COUNT(*) FROM dbo.Prescription_Summary;
  
```

Results		Messages
	TableName	RowCount
1	Medical_Practice	60
2	Drugs	6148
3	Prescriptions	116815
4	Prescription_Summary	116815

Part 2: Queries and Analysis

2.1 Question 2: Drugs in Tablet or Capsule Form

The query returned all drugs from the Drugs table where the BNF_DESCRIPTION column contains either the word 'tablet' or 'capsule'. The LIKE operator is used with the % wildcard on both sides of each search term to match the word regardless of its position within the description. The two conditions are combined with OR so that any drug matching either form is returned.

SQL Server uses a case-insensitive collation (Latin1_General_CI_AS) by default, so the query matches 'Tablets', 'tablets', and 'TABLETS' without needing additional handling. No join is required for this query as all the information comes from a single table.

```
SELECT BNF_CODE, CHEMICAL_SUBSTANCE_BNF_DESCR,  
BNF_DESCRIPTION, BNF_CHAPTER_PLUS_CODE  
FROM dbo.Drugs  
WHERE BNF_DESCRIPTION LIKE '%tablet%'  
OR BNF_DESCRIPTION LIKE '%capsule%';
```

	BNF_CODE	CHEMICAL_SUBSTANCE_BNF_DESCR	BNF_DESCRIPTION	BNF_CHAPTER_PLUS_CODE
1	0101010IAAAEAE	Magnesium oxide	Magnesium oxide 100mg tablets	01: Gastro-Intestinal System
2	0101010IBEAAC	Magnesium oxide	Oromag 160 capsules	01: Gastro-Intestinal System
3	0101010R0AAAAA	Simeticone	Simeticone 100mg capsules	01: Gastro-Intestinal System
4	0101010R0AAHAH	Simeticone	Simeticone 125mg capsules	01: Gastro-Intestinal System
5	0101010R0BHAAAA	Simeticone	WindSetlers 100mg capsules	01: Gastro-Intestinal System
6	010102100BBAQAF	Compound alginates and proprietary indigestion p...	Rennie Peppermint chewable tablets	01: Gastro-Intestinal System
7	010102100BBARAF	Compound alginates and proprietary indigestion p...	Rennie Spearmint chewable tablets	01: Gastro-Intestinal System
8	0101021B0BEAQAP	Alginic acid compound preparations	Gaviscon Advance Mint chewable tablets (Reckitt ...	01: Gastro-Intestinal System
9	0101021B0BEARA0	Alginic acid compound preparations	Gaviscon Double Action chewable tablets mint	01: Gastro-Intestinal System
10	0101021B0BEAUAR	Alginic acid compound preparations	Gaviscon Peppermint chewable tablets	01: Gastro-Intestinal System
11	0101021B0BEAWAR	Alginic acid compound preparations	Gaviscon Strawberry chewable tablets	01: Gastro-Intestinal System
12	0101021C0AAAFAP	Calcium carbonate	Calcium carbonate 500mg chewable tablets	01: Gastro-Intestinal System
13	0101021C0BIABAF	Calcium carbonate	Rennie Orange 500mg chewable tablets	01: Gastro-Intestinal System
14	0102000A0AAAAAA	Alverine citrate	Alverine 60mg capsules	01: Gastro-Intestinal System
15	0102000A0AAABAB	Alverine citrate	Alverine 120mg capsules	01: Gastro-Intestinal System
16	0102000A0BBAAAA	Alverine citrate	Spasmonal 60mg capsules	01: Gastro-Intestinal System
17	0102000A0BBABAB	Alverine citrate	Spasmonal Forte 120mg capsules	01: Gastro-Intestinal System
18	0102000AJAAAAAA	Alverine citrate/simeticone	Alverine 60mg / Simeticone 300mg capsules	01: Gastro-Intestinal System
19	0102000AJBBAAAA	Alverine citrate/simeticone	SimAlvia 60mg/300mg capsules	01: Gastro-Intestinal System

2.2 Question 3: Total Quantity per Prescription

The brief defines total quantity as the number of items (packs) multiplied by the quantity (units per pack). Since some quantities in the dataset are not integer values, the client requested the result be rounded to the nearest integer. The ROUND function handles the rounding, and CAST converts the result to INT for a clean integer output. Two columns are provided in the output: TOTAL_QUANTITY_ROUNDED shows the result of ROUND which still returns a FLOAT type, and TOTAL_QUANTITY_INT shows the same value cast to an integer. This demonstrates awareness that ROUND alone does not change the data type of the result.

```
SELECT PRESCRIPTION_CODE, PRACTICE_CODE, BNF_CODE,
ITEMS, QUANTITY,
ROUND(ITEMS * QUANTITY, 0) AS TOTAL_QUANTITY_ROUNDED,
CAST(ROUND(ITEMS * QUANTITY, 0) AS INT) AS TOTAL_QUANTITY_INT
FROM dbo.Prescriptions;
```

	PRESCRIPTION_CODE	PRACTICE_CODE	BNF_CODE	ITEMS	QUANTITY	TOTAL_QUANTITY_ROUNDED	TOTAL_QUANTITY_INT
1	0	P82034	0205051L0AAABAB	11	28	308	308
2	1	P82034	0205051L0AAABAB	4	56	224	224
3	2	P82034	0205051L0AAACAC	12	28	336	336
4	3	P82034	0205051L0AAACAC	1	84	84	84
5	4	P82034	0205051L0AAACAC	10	56	560	560
6	5	P82034	0205051L0AAADAD	2	84	168	168
7	6	P82034	0205051L0AAADAD	4	112	448	448
8	7	P82034	0205051L0AAADAD	12	56	672	672
9	8	P82034	0205051L0AAADAD	9	28	252	252
10	9	P82034	0205051M0AAAAAA	1	30	30	30
11	10	P82034	0205051M0AAAAAA	1	28	28	28
12	11	P82034	0205051M0AAAAAA	1	56	56	56
13	12	P82034	0205051M0AAABAB	3	30	90	90
14	13	P82034	0205051M0AAAFAF	2	28	56	56
15	14	P82034	0205051M0AAAFAF	1	84	84	84
16	15	P82034	0205051M0AAAFAF	1	30	30	30
17	16	P82034	0205051R0AAAAAA	2	56	112	112
18	17	P82034	0205051R0AAAAAA	4	28	112	112
19	18	P82034	0205051R0AAABAB	8	28	224	224

2.3 Question 4: Most Prescribed Chemical Substance per Month

The most frequently prescribed chemical substance for each month was identified with this query. A simple GROUP BY with MAX(COUNT) would return the count but not the associated substance name, so a more sophisticated approach was needed. The solution uses a Common Table Expression (CTE) combined with the ROW_NUMBER() window function.

The CTE first counts the number of prescription records for each combination of month and chemical substance. ROW_NUMBER() then assigns a rank within each month (partitioned by REPORT_MONTH), ordered by the prescription count in descending order. The outer query selects only the rows where RowNum equals 1, giving us the top substance for each month. The DISTINCT keyword is included as specified in the brief.

The query joins Prescriptions_Summary (for the month), Prescriptions (to link to drugs), and Drugs (for the chemical substance name). This three-table join is necessary because the month information exists in the summary table while the chemical substance information exists in the Drugs table, with Prescriptions acting as the link between them.

```
WITH MonthlySubstanceCounts AS (
SELECT ps.REPORT_MONTH,
d.CHEMICAL_SUBSTANCE_BNF_DESCR,
COUNT(*) AS PrescriptionCount,
ROW_NUMBER() OVER (
PARTITION BY ps.REPORT_MONTH
```



```

ORDER BY COUNT(*) DESC
) AS RowNum
FROM dbo.Prescription_Summary ps
INNER JOIN dbo.Prescriptions p ON ps.PRACTICE_CODE =
p.PRACTICE_CODE
INNER JOIN dbo.Drugs d ON p.BNF_CODE = d.BNF_CODE
GROUP BY ps.REPORT_MONTH, d.CHEMICAL_SUBSTANCE_BNF_DESCR
)
SELECT DISTINCT REPORT_MONTH,
CHEMICAL_SUBSTANCE_BNF_DESCR AS MostPrescribedSubstance,
PrescriptionCount
FROM MonthlySubstanceCounts
WHERE RowNum = 1
ORDER BY REPORT_MONTH;

```

	REPORT_MONTH	MostPrescribedSubstance	PrescriptionCount
1	April	Colecalciferol	544128
2	August	Colecalciferol	531391
3	December	Colecalciferol	536318
4	February	Colecalciferol	522695
5	January	Colecalciferol	532701
6	July	Colecalciferol	540970
7	June	Colecalciferol	548760
8	March	Colecalciferol	530834
9	May	Colecalciferol	550670
10	November	Colecalciferol	542008
11	October	Colecalciferol	535650
12	September	Colecalciferol	540181

2.4 Question 5: Prescriptions per BNF Chapter with Cost Statistics

This query groups prescriptions by their BNF chapter classification (BNF_CHAPTER_PLUS_CODE from the Drugs table) and calculates four aggregate measures for each chapter: the total number of prescriptions (COUNT), the average cost (AVG), the minimum cost (MIN), and the maximum cost (MAX). The ROUND function is applied to the average for readability.

The query uses an INNER JOIN between Prescriptions and Drugs on BNF_CODE, since the chapter classification is stored in the Drugs table while the cost data is in the Prescriptions table. Results are ordered by the number of prescriptions in descending order, making it easy to identify which therapeutic areas generate the most prescribing activity in Bolton.

```

SELECT d.BNF_CHAPTER_PLUS_CODE,
COUNT(p.PRESCRIPTION_CODE) AS NumberOfPrescriptions,
ROUND(AVG(p.ACTUAL_COST), 2) AS AverageCost,
ROUND(MIN(p.ACTUAL_COST), 2) AS MinimumCost,
ROUND(MAX(p.ACTUAL_COST), 2) AS MaximumCost

```

```

FROM dbo.Prescriptions p
INNER JOIN dbo.Drugs d ON p.BNF_CODE = d.BNF_CODE
GROUP BY d.BNF_CHAPTER_PLUS_CODE
ORDER BY NumberOfPrescriptions DESC;

```

	BNF_CHAPTER_PLUS_CODE	NumberOfPrescriptions	AverageCost	MinimumCost	MaximumCost
1	04: Central Nervous System	28866	28.4	0.13	2765.62
2	02: Cardiovascular System	19186	36	0.13	11094.75
3	06: Endocrine System	12462	61.17	0.17	3490.7
4	01: Gastro-Intestinal System	8777	35.13	0.14	2777.69
5	09: Nutrition and Blood	7944	41.82	0.14	4250.44
6	03: Respiratory System	7057	75.87	0.15	4161.81
7	21: Appliances	5856	38.35	0.21	2552.92
8	13: Skin	5692	22.25	0.49	1402.48
9	05: Infections	4657	21.25	0.2	1262.21
10	07: Obstetrics, Gynaecology and Urinary-Tract Di...	3999	25.46	0.15	803.49
11	10: Musculoskeletal and Joint Diseases	3634	16.25	0.27	1476.64
12	11: Eye	2676	21.44	1.34	434.8
13	23: Stoma Appliances	1697	82.26	1.59	882.53
14	12: Ear, Nose and Oropharynx	1274	23.68	1.68	315.78
15	20: Dressings	1014	40.22	0.22	2599.19
16	08: Malignant Disease and Immunosuppression	754	54.94	0.29	2197.16
17	22: Incontinence Appliances	535	42.87	2.87	538.89
18	19: Other Drugs and Preparations	338	32.25	0.43	1193.35
19	15: Anaesthesia	271	35.67	0.28	345.25

2.5 Question 6: Most Expensive Prescription per Practice (Over £4,000)

This query finds the single most expensive prescription for each practice and returns only those where the cost exceeds £4,000. The approach uses a correlated subquery: the inner SELECT finds the maximum ACTUAL_COST for the same practice, and the outer query retrieves the full details of prescriptions matching that maximum. The additional condition p.ACTUAL_COST > 4000 filters out practices whose most expensive prescription falls below the threshold.

The query joins three tables: Prescriptions for the cost data, Medical_Practice for the practice name (as required by the brief), and Drugs for the drug description. Results are sorted in descending order by prescription cost.

```

SELECT mp.PRACTICE_NAME, p.PRACTICE_CODE,
p.PRESCRIPTION_CODE, d.BNF_DESCRIPTION,
p.ACTUAL_COST AS PrescriptionCost
FROM dbo.Prescriptions p
INNER JOIN dbo.Medical_Practice mp ON p.PRACTICE_CODE =
mp.PRACTICE_CODE
INNER JOIN dbo.Drugs d ON p.BNF_CODE = d.BNF_CODE
WHERE p.ACTUAL_COST = (
SELECT MAX(p2.ACTUAL_COST)
FROM dbo.Prescriptions p2
WHERE p2.PRACTICE_CODE = p.PRACTICE_CODE
)

```

AND p.ACTUAL_COST > 4000
ORDER BY p.ACTUAL_COST DESC;

	PRACTICE_NAME	PRACTICE_CODE	PRESCRIPTION_CODE	BNF_DESCRIPTION	PrescriptionCost
1	UNSWORTH GROUP PRACTICE	P82015	109713	Adjuvanted quadrivalent flu vacc (SA, inact) inj 0....	21570.919921875
2	BROMLEY MEADOWS SURGERY	P82643	30914	Adjuvanted quadrivalent flu vacc (SA, inact) inj 0....	11031.5888671875
3	KILDONAN HOUSE	P82003	37418	Supemtek Quadrivalent vaccine (recombinant) inj ...	8659.595703125
4	KEARSLEY MEDICAL CENTRE	P82007	18405	Apixaban 5mg tablets	7313.5810546875
5	BOLTON COMMUNITY PRACTICE	Y03079	95216	Apixaban 5mg tablets	6069.77490234375
6	HARWOOD MEDICAL CENTRE	P82016	11263	Apixaban 5mg tablets	4626.95947265625
7	MANDALAY MEDICAL CENTRE	P82023	43401	Apixaban 5mg tablets	4377.65234375
8	CROMPTON VIEW SURGERY	P82607	78135	Elemental 028 Extra powder (3 flavours)	4250.4384765625
9	DALEFIELD SURGERY	P82010	79815	Fluad Tetra vaccine inj 0.5ml pre-filled syringes	4031.95727539063

2.6 Question 7: Additional Queries

7a: Practice Specialisation Analysis

Use case: Useful for evaluating practice specialisation.

Techniques used: EXISTS, INNER JOIN, GROUP BY, HAVING, ORDER BY, COUNT DISTINCT.

This query identifies medical practices that prescribe from a limited number of BNF chapters (five or fewer), which may indicate that the practice has a degree of clinical specialisation rather than being a general practice. The EXISTS predicate is used to confirm that each practice has at least one prescription on record before it is included in the results. GROUP BY aggregates prescriptions at the practice level, and HAVING filters to only those practices with five or fewer distinct chapter codes.

From a commercial perspective, this information is valuable for the pharmaceutical company because specialist practices are more likely to be high-volume purchasers within their specific therapeutic area. Targeted marketing of specialist drugs to these practices would be more effective than a blanket approach.

```

SELECT
    mp.PRACTICE_CODE,
    mp.PRACTICE_NAME,
    COUNT(DISTINCT d.BNF_CHAPTER_PLUS_CODE) AS ChaptersServed,
    COUNT(p.PRESCRIPTION_CODE) AS TotalPrescriptions
FROM dbo.Medical_Practice mp
INNER JOIN dbo.Prescriptions p ON mp.PRACTICE_CODE = p.PRACTICE_CODE
INNER JOIN dbo.Drugs d ON p.BNF_CODE = d.BNF_CODE
WHERE EXISTS (
    SELECT 1 FROM dbo.Prescriptions px
    WHERE px.PRACTICE_CODE = mp.PRACTICE_CODE
)
GROUP BY mp.PRACTICE_CODE, mp.PRACTICE_NAME
HAVING COUNT(DISTINCT d.BNF_CHAPTER_PLUS_CODE) <= 5
ORDER BY ChaptersServed ASC, TotalPrescriptions DESC;

```

Results		Messages		
	PRACTICE_CODE	PRACTICE_NAME	ChaptersServed	TotalPrescriptions
1	Y03641	ACHIEVE BOLTON RECOVERY SERVICE	1	212
2	Y00215	ORTHOPAEDIC & RHEUMATOLOGY	1	2
3	P82654	BOLTON HOSPICE	1	1
4	Y00409	NEURO-REHAB SERVICE	1	1

7b: Bulk Purchasing Opportunities

Use case: Can support bulk purchasing decisions.

Techniques used: IN (subquery), GROUP BY, HAVING, ORDER BY, SUM, COUNT DISTINCT, ROUND, CAST.

This query identifies drugs that are prescribed in high volumes across at least 10 different practices, suggesting widespread demand suitable for bulk purchasing negotiations. The IN subquery first filters to only those BNF codes prescribed by 10 or more distinct practices. The outer query then aggregates the total items, total units (items multiplied by quantity, rounded to the nearest integer), and total cost for each qualifying drug.

The pharmaceutical company could use these results to negotiate volume discounts with suppliers for the most widely prescribed medications in Bolton, potentially reducing procurement costs significantly across the region.

```

SELECT
    d.BNF_CODE,
    d.CHEMICAL_SUBSTANCE_BNF_DESCR,
    d.BNF_DESCRIPTION,
    COUNT(DISTINCT p.PRACTICE_CODE) AS NumberOfPractices,
    SUM(p.ITEMS) AS TotalItemsPrescribed,
    CAST(ROUND(SUM(p.ITEMS * p.QUANTITY), 0) AS INT) AS TotalUnitsPrescribed,
    ROUND(SUM(p.ACTUAL_COST), 2) AS TotalCost
FROM dbo.Drugs d
INNER JOIN dbo.Prescriptions p ON d.BNF_CODE = p.BNF_CODE
WHERE d.BNF_CODE IN (
    SELECT p2.BNF_CODE
    FROM dbo.Prescriptions p2
    GROUP BY p2.BNF_CODE
    HAVING COUNT(DISTINCT p2.PRACTICE_CODE) >= 10
)
GROUP BY d.BNF_CODE, d.CHEMICAL_SUBSTANCE_BNF_DESCR, d.BNF_DESCRIPTION
HAVING SUM(p.ITEMS) > 100
ORDER BY TotalUnitsPrescribed DESC;

```

Results		Messages						
	BNF_CODE	CHEMICAL_SUBSTANCE_BNF_DESCR	BNF_DESCRIPTION	NumberOfPractices	TotalItemsPrescribed	TotalUnitsPrescribed	TotalCost	
1	090402000BBRRA0	Enteral nutrition	Ensure Plus milkshake style liquid (9 flavours)	45	244	2152200	11875.12	
2	090402000BBVTA0	Enteral nutrition	Ensure Compact liquid (4 flavours)	44	175	1065750	11123.57	
3	0407010H0AAAMAM	Paracetamol	Paracetamol 500mg tablets	51	7044	929568	20309.24	
4	0407010F0BSAAAH	Co-codamol (Codeine phosphate/paracetamol)	Zapain 30mg/500mg tablets	52	6933	805638	23773.48	
5	0106040G0AAAAAA	Lactulose	Lactulose 3.1-3.7g/5ml oral solution	51	1275	702040	3795.54	
6	130201000BBICBW	Other emollient preparations	Demol 500 lotion	52	1077	652500	7382.89	
7	0103050P0AAAAAA	Omeprazole	Omeprazole 20mg gastro-resistant capsules	52	16871	651122	56886.04	
8	0601022B0AAABAB	Metformin hydrochloride	Metformin 500mg tablets	52	5939	589596	14086.78	
9	0212000B0AAABAB	Atorvastatin	Atorvastatin 20mg tablets	52	11449	402962	12087.06	
10	130201000BBJGCA	Other emollient preparations	Doublebase gel	52	619	359400	4051.06	
11	0101021B0BIAAAH	Alginic acid compound preparations	Peptac liquid aniseed	49	544	313050	1173.44	
12	21220000233	Emollients	Cetaben cream	51	495	303350	3645.54	
13	0101021B0BIABAH	Alginic acid compound preparations	Peptac liquid peppermint	49	509	299810	1123.03	
14	0206020A0AAAAAA	Amlodipine	Amlodipine 5mg tablets	51	7946	288904	6535.94	
15	0101021B0BEAIAL	Alginic acid compound preparations	Gaviscon Advance oral suspension aniseed (Reck...	51	456	281500	3132.04	

7c: Unusual or Potentially Erroneous Prescriptions

Use case: Could report errors or unusual cases.

Techniques used: CTE, INNER JOIN, system functions (AVG, STDEV, ROUND), GROUP BY, HAVING, ORDER BY.

This query uses a statistical approach to identify outlier prescriptions. A CTE first calculates the average cost and standard deviation for each drug (filtering to drugs with more than five prescriptions and a non-zero standard deviation for statistical validity). The outer query then identifies individual prescriptions where the cost exceeds three standard deviations above the mean for that drug, calculating a Z-score to quantify how far each outlier sits from the average.

These outliers could represent data entry errors (for example, a quantity entered incorrectly), unusual clinical cases requiring exceptionally large quantities, or pricing anomalies. The Z-score column allows the pharmaceutical company to prioritise investigation, starting with the most extreme outliers. This kind of quality assurance query is particularly important in healthcare data where errors can have both financial and clinical implications.

```

WITH DrugStats AS (
    SELECT
        BNF_CODE,
        AVG(ACTUAL_COST) AS AvgCost,
        STDEV(ACTUAL_COST) AS StdDevCost
    FROM dbo.Prescriptions
    GROUP BY BNF_CODE
    HAVING COUNT(*) > 5 AND STDEV(ACTUAL_COST) > 0
)
SELECT
    p.PRESCRIPTION_CODE,
    mp.PRACTICE_NAME,
    d.BNF_DESCRIPTION,
    ROUND(p.ACTUAL_COST, 2) AS PrescriptionCost,
    ROUND(ds.AvgCost, 2) AS AverageCostForDrug,
    ROUND(ds.StdDevCost, 2) AS StdDeviation,
    ROUND((p.ACTUAL_COST - ds.AvgCost) / ds.StdDevCost, 2) AS ZScore
FROM dbo.Prescriptions p
INNER JOIN DrugStats ds ON p.BNF_CODE = ds.BNF_CODE
INNER JOIN dbo.Medical_Practice mp ON p.PRACTICE_CODE = mp.PRACTICE_CODE
INNER JOIN dbo.Drugs d ON p.BNF_CODE = d.BNF_CODE
WHERE p.ACTUAL_COST > (ds.AvgCost + 3 * ds.StdDevCost)
ORDER BY ZScore DESC;

```

	PRESCRIPTION_CODE	PRACTICE_NAME	BNF_DESCRIPTION	PrescriptionCost	AverageCostForDrug	StdDeviation	ZScore
1	32574	KEARSLEY MEDICAL CENTRE	Prednisolone 5mg tablets	185.9	6.83	12.05	14.86
2	33731	KEARSLEY MEDICAL CENTRE	Tramadol 50mg capsules	206.87	9.56	16.18	12.2
3	66803	STONEHILL MEDICAL CENTRE	Gabapentin 300mg capsules	259.46	13.58	22.68	10.84
4	79443	DALEFIELD SURGERY	Ferrous fumarate 305mg capsules	98.43	5.87	9.23	10.03
5	66805	STONEHILL MEDICAL CENTRE	Gabapentin 300mg capsules	235.62	13.58	22.68	9.79
6	110093	UNSWORTH GROUP PRACTICE	Apixaban 2.5mg tablets	6430.94	390.31	630.64	9.58
7	75058	CHARLOTTE STREET SURGERY	Ferrous sulfate 200mg tablets	104.09	9.26	10.23	9.27
8	112400	UNSWORTH GROUP PRACTICE	Non-woven fabric swab 4ply non-sterile 10cm x 10cm	57.26	4.91	5.8	9.02
9	109884	UNSWORTH GROUP PRACTICE	Docusate 100mg capsules	192.53	14.51	20.01	8.9
10	109549	UNSWORTH GROUP PRACTICE	Accrete D3 tablets	458.89	29.15	48.46	8.87
11	78503	DALEFIELD SURGERY	Bisacodyl 5mg gastro-resistant tablets	140.34	11.88	14.62	8.79

7d: Month-on-Month Comparison of Prescribing Volumes

Use case: Showing comparisons of current with previous.

Techniques used: CTE, LEFT JOIN (self-join), CASE, system functions (SUM, ROUND, ISNULL), GROUP BY, ORDER BY.

This query compares prescribing volumes between two consecutive months for each practice, calculating an absolute change and then a percentage change in total items prescribed. A CTE first aggregates the Prescriptions_Summary data to produce monthly totals per practice. The outer query then performs a self-join on the CTE using a CASE expression to map each month to its predecessor (for example, February maps to January). A LEFT JOIN is used rather than an INNER JOIN to ensure that practices appearing for the first time in a given month are still included, with ISNULL handling the NULL values that result from the absence of a previous month.

The percentage change column allows the pharmaceutical company to quickly identify practices with significant increases or decreases in prescribing activity. A sharp increase might indicate a local health concern or a new prescriber at the practice, while

a significant decrease could suggest the practice has switched to an alternative drug. Both scenarios are commercially relevant for planning supply chains and informing sales strategy.

```
WITH MonthlySummary AS (  
    SELECT  
        PRACTICE_CODE,  
        REPORT_MONTH,  
        SUM(TOTAL_ITEMS) AS MonthlyItems,  
        ROUND(SUM(TOTAL_COST), 2) AS MonthlyCost  
    FROM dbo.Prescription_Summary  
    GROUP BY PRACTICE_CODE, REPORT_MONTH  
)  
SELECT  
    curr.PRACTICE_CODE,  
    mp.PRACTICE_NAME,  
    curr.REPORT_MONTH AS CurrentMonth,  
    prev.REPORT_MONTH AS PreviousMonth,  
    curr.MonthlyItems AS CurrentItems,  
    ISNULL(prev.MonthlyItems, 0) AS PreviousItems,  
    curr.MonthlyItems - ISNULL(prev.MonthlyItems, 0) AS ItemChange,  
    CASE  
        WHEN prev.MonthlyItems IS NULL OR prev.MonthlyItems = 0 THEN NULL  
        ELSE ROUND((((curr.MonthlyItems - prev.MonthlyItems) * 100.0  
            / prev.MonthlyItems), 1)  
    END AS PercentChange,  
    curr.MonthlyCost AS CurrentCost,  
    ISNULL(prev.MonthlyCost, 0) AS PreviousCost  
FROM MonthlySummary curr  
LEFT JOIN MonthlySummary prev  
    ON curr.PRACTICE_CODE = prev.PRACTICE_CODE  
    AND prev.REPORT_MONTH = CASE curr.REPORT_MONTH  
        WHEN 'February' THEN 'January'  
        WHEN 'March' THEN 'February'  
        WHEN 'April' THEN 'March'  
        WHEN 'May' THEN 'April'  
        WHEN 'June' THEN 'May'  
        WHEN 'July' THEN 'June'  
        WHEN 'August' THEN 'July'  
        WHEN 'September' THEN 'August'  
        WHEN 'October' THEN 'September'  
        WHEN 'November' THEN 'October'  
        WHEN 'December' THEN 'November'  
        ELSE NULL  
    END  
INNER JOIN dbo.Medical_Practice mp ON curr.PRACTICE_CODE = mp.PRACTICE_CODE  
WHERE curr.REPORT_MONTH = 'August' -- This can be changed to compare different months  
ORDER BY PercentChange DESC;
```

Results		Messages								
	PRACTICE_CODE	PRACTICE_NAME	CurrentMonth	PreviousMonth	CurrentItems	PreviousItems	ItemChange	PercentChange	CurrentCost	PreviousCost
1	Y03641	ACHIEVE BOLTON RECOVERY SERVICE	August	July	330	41	289	704.9000000000000	2427.53	362.35
2	00T998	UNIDENTIFIED DOCTORS	August	July	3	1	2	200.0000000000000	32.43	1.29
3	Y07546	TURTON PCN HUB	August	July	23	11	12	109.1000000000000	278.93	29.16
4	P82660	DEANE CLINIC 1	August	July	565	334	231	69.2000000000000	2821.8	2197.53
5	P82015	UNSWORTH GROUP PRACTICE	August	July	3998	2539	1459	57.5000000000000	29225.67	22277.41
6	P82029	HALLIWELL SURGERY 2	August	July	676	444	232	52.3000000000000	2697.06	2846.73
7	Y04600	BARDOC GP OOH	August	July	183	122	61	50.0000000000000	606.67	355.12
8	Y05353	WEEKEND SERVICE OOH	August	July	82	57	25	43.9000000000000	547.36	291.48
9	P82005	STABLE FOLD SURGERY	August	July	822	589	233	39.6000000000000	11188.71	7603.9
10	P82652	FARNWORTH FAMILY PRACTICE	August	July	764	553	211	38.2000000000000	3903.93	3885.87
11	P82016	HARWOOD MEDICAL CENTRE	August	July	2125	1595	530	33.2000000000000	12389.18	16237.18
12	P82033	BRADFORD STREET SURGERY	August	July	367	277	90	32.5000000000000	2296.43	3207.95
13	P82022	HALLIWELL SURGERY 1	August	July	952	739	213	28.8000000000000	6765.88	5760.71
14	Y00186	3D MEDICAL CENTRE	August	July	200	158	42	26.6000000000000	1482.65	1286.75
15	P82001	THE DUNSTAN PARTNERSHIP	August	July	1775	1440	335	23.3000000000000	11474.68	11118.27

August – July

Results		Messages								
	PRACTICE_CODE	PRACTICE_NAME	CurrentMonth	PreviousMonth	CurrentItems	PreviousItems	ItemChange	PercentChange	CurrentCost	PreviousCost
1	P82030	DEANE MEDICAL CENTRE	February	January	691	384	307	79.9000000000000	4931.56	3800.73
2	P82011	TONGE FOLD HEALTH CENTRE	February	January	1221	754	467	61.9000000000000	9551.78	5822.74
3	Y03366	OLIVE FAMILY PRACTICE	February	January	566	360	206	57.2000000000000	3762.87	3507.05
4	Y03641	ACHIEVE BOLTON RECOVERY SERVICE	February	January	312	199	113	56.8000000000000	1826.35	2443.43
5	Y02790	BOLTON MEDICAL CENTRE	February	January	916	589	327	55.5000000000000	6259.09	5811.77
6	P82003	KILDONAN HOUSE	February	January	2540	1754	786	44.8000000000000	17030.03	13173.38
7	P82627	CORNERSTONE SURGERY	February	January	598	428	170	39.7000000000000	4996.29	3427.87
8	P82002	PIKES LANE 1	February	January	1735	1277	458	35.9000000000000	11436.46	9239.92
9	P82005	STABLE FOLD SURGERY	February	January	812	623	189	30.3000000000000	8251.83	7396.32
10	P82660	DEANE CLINIC 1	February	January	566	438	128	29.2000000000000	1961.4	3246.01
11	Y00186	3D MEDICAL CENTRE	February	January	226	175	51	29.1000000000000	1746.48	2092.84
12	P82009	ST HELENS ROAD PRACTICE	February	January	1264	1002	262	26.1000000000000	8250.22	10621.02
13	P82001	THE DUNSTAN PARTNERSHIP	February	January	2087	1673	414	24.7000000000000	14172.62	15603.08
14	P82634	WYRESDALE ROAD SURGERY	February	January	533	429	104	24.2000000000000	4130.42	4380
15	Y05353	WEEKEND SERVICE OOH	February	January	91	74	17	23.0000000000000	509.87	368.62

Feb-Jan

Conclusions

This report demonstrated the process of creating a relational database from four CSV files of NHS prescribing data and using T-SQL queries to extract meaningful insights from the data. The PrescriptionsDB database was created with appropriate primary and foreign key constraints to enforce referential integrity between the four tables.

The six required queries addressed a range of analytical needs, from simple filtering (tablets and capsules) and arithmetic calculations (total quantities) through to more complex aggregation with window functions (most prescribed substance per month) and correlated subqueries (most expensive prescription per practice). Each query was designed to answer a specific business question that a pharmaceutical company operating in the Bolton area would find commercially relevant.

The four additional queries extended the analysis further, demonstrating the use of EXISTS, IN with subqueries, statistical functions (STDEV for outlier detection), and self-joins for temporal comparisons. Together, these queries showcase a range of T-SQL techniques including joins, subqueries, CTEs, window functions, aggregate functions, CASE expressions, and system functions, applied to a real-world dataset to derive actionable insights for data-driven decision making.

References

- Connolly, T. and Begg, C. (2015) Database Systems: A Practical Approach to Design, Implementation, and Management. 6th edn. Harlow: Pearson Education.
- Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems. 7th edn. Hoboken: Pearson.
- Microsoft (2024) SQL Server Technical Documentation. Available at: <https://learn.microsoft.com/en-us/sql/sql-server/> (Accessed: 10 April 2026).
- Ramakrishnan, R. and Gehrke, J. (2003) Database Management Systems. 3rd edn. New York: McGraw-Hill.
- Stallings, W. (2017) Cryptography and Network Security: Principles and Practice. 7th edn. Harlow: Pearson.